

Institut National des Sciences Appliquees et de Technologie

UNIVERSITE DE CARTHAGE

Projet de Fin d'Etudes

Filiere : **RT**

Deep Learning pour l'imagerie et diagnostic medical

Presente par

BEN GAMRA Miniar

Encadrant INSAT : **Mr BEN SALAH Med Bassem**
Encadrant ENTREPRISE : **Mr AKHLOUFI Moulay**

JURY

M. President (President)
Mme. Rapporteur (Rapporteur)

Annee Universitaire : 2018/2019

Remerciements

Je tiens à remercier toute l'équipe PRIME pour leurs aides, leurs précieux conseils et tous les agréables moments passés auprès d'eux.

Particulièrement, je témoigne une sincère reconnaissance à mon superviseur à l'Université de Moncton Mr AKHLOUFI Moulay qui m'a accordé sa confiance pour intégrer son équipe et qui a toujours été à l'écoute et m'a procuré des conseils judicieux. Une attention particulière revient à COUTURIER Andy pour le temps qu'il a consacré pour m'aider.

Je tiens également à remercier mon encadrant à l'INSAT Mr BEN SALAH Med Bassem pour m'avoir accompagné et guidé tout au long de mon stage.

Une pensée chaleureuse va à tous les stagiaires et les étudiants en master.

Enfin, j'adresse mes remerciements les plus chers à ceux qui ont suivi de près ou de loin l'ensemble des tribulations survenues au cours de mon stage, à ma famille et à mes amis.

Table des Matières

Liste des Figures **v**

Liste des Tableaux **vii**

Résumé **viii**

Abstract **ix**

Introduction Générale **1**

I Contexte du projet **3**

1 Présentation de l'université d'accueil 3

2 Cancer du poumon 4

2.1 Définition 4

2.2 Statistiques 4

2.3 Les méthodes classiques pour la détection du cancer du poumon 4

2.4 Problématique 5

3 Le perceptron multi-couches 6

3.1 Perceptron simple 6

3.2 Perceptron multi-couches 7

3.3 L'apprentissage 7

3.4 La descente de gradient 8

3.5 Métriques 9

4 Deep Learning . . . 9
5 Les réseaux de neurones convolutifs . . . 10
5.1 Définition . . . 10
5.2 Architecture générale . . . 10
5.3 Les couches . . . 11
5.4 Avantages . . . 12
5.5 Les paramètres . . . 13
6 Présentation du projet . . . 13
6.1 Classification des tumeurs . . . 13
6.2 Planification du projet . . . 14

II Etude de l'existant **15**
1 Les approches 2D . . . 15
1.1 CNN/ELM . . . 16
1.2 Multi-view CNN . . . 17
1.2.1 Extraction des images candidates . . . 18
1.2.2 Extraction des tranches . . . 19
1.2.3 Configuration de réseau 2D-CNN . . . 19
1.2.4 Fusion des flux ConvNets . . . 19
2 Les approches 3D . . . 20

III Optimisation du calcul et du traitement **23**
1 Résultats possibles . . . 23
2 La régularisation . . . 24
2.1 L2-régularisation (Weight decay) . . . 25
2.2 Dropout . . . 25
2.3 Early stopping . . . 26
2.4 Augmentation des données . . . 27
2.4.1 Phase de prétraitement . . . 27
2.4.2 Analyse des données . . . 27
2.4.3 Equilibrage de la base de données . . . 27
2.4.4 Augmentation des données . . . 27
3 Optimisation des performances de calcul . . . 28
3.1 Le GPU . . . 28
3.1.1 Entrainer le réseau sur plusieurs GPUs sur une seule machine . 28

3.1.2 Entrainer le réseau sur plusieurs GPUs multi-machines 29
3.2 Génération des données 29
3.2.1 Data Generator 29
3.2.2 LRU cache 30
4 Classification des données déséquilibrées 30
4.1 Stratégies d'échantillonnage 31
4.2 Stratégies algorithmiques 31

IV Implémentation et discussion des résultats **33**
1 Outils utilisés 33
2 Configuration matérielle 34
3 Base de données 35
4 Réalisation du modèle 35
4.1 Préparation des données 35
4.2 Equilibrage des données 36
4.3 Architecture du réseau 36
4.4 Initialisation des paramètres 38
4.5 La fonction de perte 39
4.6 Optimisation du calcul 39
4.7 Optimisation du traitement 39
5 Tests et résultats 40
5.1 Données en entrée 40
5.2 Taux d'apprentissage 40
5.3 Dropout 42
5.4 Résultat final 42

Conclusion Générale et Perspectives **44**

Bibliographiques **46**

Liste des Figures

I.1 Logo Université de Moncton 4
I.2 (a) : Pourcentage de tous les nouveaux cas de cancer estimés en 2017 chez les deux sexes combinés, (b) : Pourcentage de tous les décès de cancer estimés en 2017 chez les deux sexes combinés 5
I.3 Exemples des tumeurs pulmonaires : Le rectangle vert présente des tumeurs malignes et le rouge présente des tumeurs bénignes [1] 6
I.4 Modèle du perceptron simple 7
I.5 Architecture du CNN 11
I.6 Diagramme de Gantt 14

II.1 Architecture de SAE 16
II.2 Architecture de MSAE 17
II.3 Architecture de l'approche Multi-view CNN 18
II.4 Architecture de l'approche multi-level contexual CNN 21

III.1 Les cas de figure possible lors de la phase d'apprentissage 24
III.2 L'effet de lambda sur l'apprentissage 25
III.3 Dropout 26
III.4 Early Stopping 26
III.5 Répartition des calculs entre le CPU et le GPU 28
III.6 Parallélisme multi-GPUs 29
III.7 L'effet du gamma sur la fonction Focal-Loss 32

IV.1 Aperçu de l'environnement de Deep Learning 34
IV.2 Des tranches 2D extraites de différentes localisations d'un seul patient [2] 36
IV.3 Architecture du modèle 3D-DCNN 37
IV.4 Configuration du modèle 3D-DCNN pour une image de taille 32*32*32 38
IV.5 Modèle1 avec des images en entrée de taille 32*323*32 ; (b) Modèle2 avec des images en entrée de taille 64*64*64 40
IV.6 lambda=0.0001 41
IV.7 Taux d'apprentissage=0.00001 41
IV.8 Modèle sans dropout 42
IV.9 Modèle avec Dropout 43

Liste des Tableaux

II.1 Architecture de l'approche Multi-contextual 3D-CNN 21

Résumé

Le cancer du poumon est la forme de cancer la plus couramment diagnostiquée au Canada, d'après la société canadienne du cancer. C'est la principale cause de décès par cancer tant chez l'homme que chez la femme au Canada. Sa détection précoce est très importante puisqu'elle permet d'accélérer la procédure de traitement et par la suite diminuer le taux de mortalité et sauver des vies. Cependant, ce processus n'est pas évident en raison du nombre considérable de tomodensitogrammes à analyser, les expertises nécessaires pour se faire et l'inexistence d'une formule claire pour décrire une tumeur. Les techniques d'intelligence artificielle peuvent être utiles dans ces cas pour offrir un système capable de classifier automatiquement les tumeurs et accélérer le processus de diagnostic tout en gardant un faible taux d'erreur. Dans cette étude, nous avons mettre en oeuvre un algorithme qui permet d'exploiter les tomodensitogrammes en 3D pour étiqueter les tumeurs. Nous avons utilisé la base de données LUNA16 pour obtenir une performance de 97% de AUC.

Abstract

Lung cancer is considered the deadliest cancer worldwide with around 1.6 millions deaths every year, according to the CCS (Canadian Cancer Society). Early detection of lung cancer is very important in healthcare as it can help save lives and decrease the cancer mortality rate. Due to the huge number of CT scans that need to be analyzed annually, the detection of lung cancer at early stages become a tedious task for radiologists.

Artificial intelligence techniques can be useful in developing a solution for this problem by offering a system that can automatically classify tumors and speed-up the diagnosis process while making the result more repeatable and accurate.

Since CT scans and nodules are 3D in nature, we implemented a deep learning algorithm based on the use of 3D convolutional neural networks. This network will be used to classify the 3D nodules as benign or malignant. To develop the algorithm, we used a dataset provided by the Lung Nodule Analysis 2016 challenge (Luna 16) and obtained an overall AUC (Area Under Curve) performance of 97%.

Introduction Générale

L'intelligence artificielle est un domaine qui a été développé après la seconde guerre mondiale dont le but est de créer des machines capables d'apprendre et de s'améliorer d'une manière autonome. Cette technologie se révèle être utile dans toutes les tâches intellectuelles.
Ce concept s'est ensuite étendu pour englober plusieurs sous domaines dont la classification des images qui a eu un progrès considérable au cours de ses dernières années. Ce progrès est dû aux nombreux travaux dans ce domaine et à la disponibilité des bases d'images internationales qui ont permis aux chercheurs de signaler l'exécution de leurs approches.

En effet, cette technologie fait apparaitre un type de réseau plus particulier que le réseau neuronal multicouche dont l'architecture des connexions est inspirée de celle du cortex visuel. Ce réseau est appelé le réseau de neurone convolutif. Dans les années 80, Yan le Cun a créé ce réseau mais il n'a pas pu le mettre en œuvre au regard de la puissance de calcul disponible à l'époque et du volume de données à traiter. En fait, les réseaux de neurones convolutifs et les réseaux neuronaux en général ont été délaissés par la communauté de recherche entre 1997 et 2012.

En 2011 et 2012 trois événements ont changé la situation. Tout d'abord, les GPU (GraphicalProcessing Unit) capables de calculer plus que mille milliards d'opérations par seconde sont devenus disponibles pour un prix moins cher. Ces processeurs spécialisés, initialement conçus pour le rendu graphique des jeux vidéo, se sont avérés être très performants pour les calculs des réseaux neuronaux. Deuxièmement, des expériences menées simultanément à Microsoft, Google et IBM avec l'aide du laboratoire de Geoff Hinton ont montré que les réseaux profonds pouvaient diminuer de moitié les taux d'erreurs des systèmes de reconnaissance vocale. Troisièmement, plusieurs records en reconnaissance d'image ont été battus par des réseaux de neurones convolutifs. L'événement le plus marquant a été la victoire éclatante de l'équipe de

Toronto dans la compétition de reconnaissance d'objets «ImageNet ». La diminution des taux d'erreurs était telle qu'une véritable révolution.

Du jour au lendemain, la majorité des équipes de recherche en parole et en vision ont abandonné leurs méthodes préférées et sont passées aux réseaux de neurones convolutifs et autres réseaux neuronaux et le monde a eu une explosion des données étiquetées.

Dans ce contexte s'inscrit ce projet de fin d'études intitulé « Deep Learning pour l'imagerie médicale » qui ambitionne de développer un algorithme d'apprentissage profond basé sur les réseaux de neurones convolutifs. Ceci permettra la classification des tumeurs pour le cancer du poumon à partir des images 3D.

Dans ce travail, nous allons utiliser des algorithmes de Machine Learning qui sont déjà appliqués au problème de classification des images en comparant les résultats de plusieurs modèles allant du plus simple au plus sophistiqué que les ressources disponibles nous permettent. Nous essayerons par la suite de faire plusieurs optimisations en faisant varier les hyper paramètres qui constituent le modèle afin de comprendre l'impact de chacun d'eux sur l'apprentissage et de choisir la valeur qui nous mène à améliorer la métrique.

Le présent rapport s'étale sur 4 chapitres. Dans le premier chapitre, nous présentons l'organisme d'accueil et le cadre du projet. Dans le deuxième chapitre, nous entamons une étude sur l'existant. Le troisième chapitre présente des méthodes utilisées pour optimiser le calcul et le traitement et le dernier chapitre est consacré pour présenter le modèle, l'architecture et les résultats obtenus.

Chapitre I

Contexte du projet

Introduction

Dans ce chapitre, nous présentons l'organisme d'accueil, suivi par des prérequis nécessaires dans notre étude. Ensuite, nous abordons la problématique qui a créé le besoin de réaliser un tel projet et nous terminons par une mise au point de la solution proposée.

1 Présentation de l'université d'accueil

L'Université de Moncton est la plus grande université canadienne de langue française à l'extérieur du Québec, reconnu pour l'excellence de ses programmes d'études et de recherche.
Fondée en 1963, l'université de Moncton est constituée de trois campus situés à Edmundston, Moncton et Shippagan, au Nouveau-Brunswick. Elle offre des programmes aux trois cycles d'études et ses services sont destinés à l'ensemble des étudiants des régions francophones du pays, devenant ainsi le symbole par excellence de la vitalité linguistique et culturelle des francophones vivant à l'extérieur du Québec. Elle accueille aussi les étudiants internationaux en provenance d'une quarantaine de pays.
L'Université de Moncton offre un enseignement personnalisé qui privilégie non seulement les échanges et l'apprentissage, mais également un environnement propice aux études. En 2007, l'Université a lancé un vaste programme d'appui à la réussite des études universitaires en offrant aux étudiantes et aux étudiants les services et les outils nécessaires à la poursuite de leurs projets d'études, tant sur le plan académique que sur le plan personnel, professionnel et social.

Mon stage de fin d'études s'est déroulé au département informatique de l'université de Moncton au sein du laboratoire de recherche PRIME du 1er Mai au 31 Juillet 2019 sous la supervision du Professeur Moulay AKHLOUFI.



Figure I.1 – Logo Université de Moncton

2 Cancer du poumon

2.1 Définition

Le cancer du poumon prend naissance dans les cellules du poumon. En effet, la tumeur cancéreuse (maligne) est un groupe de cellules cancéreuses qui peuvent envahir et détruire le tissu voisin. Elle peut aussi se propager à d'autres parties du corps.

2.2 Statistiques

D'après la société canadienne du cancer, le cancer du poumon est la forme de cancer la plus couramment diagnostiquée au Canada. C'est la principale cause de décès par cancer tant chez l'homme que chez la femme. Comme illustré dans la figure I.2, en 2017, ce type de cancer a représenté 14% de tous les nouveaux cas du cancer, 26% de tous les décès par cancer. Ainsi, les statistiques ont montré que le taux de servie nette après 5 ans du diagnostic est environ 17% [3].

2.3 Les méthodes classiques pour la détection du cancer du poumon

Habituellement, lorsque certains symptômes laissent suspecter un cancer pulmonaire, et pour réaliser un diagnostic fiable, le médecin va prescrire plusieurs examens dont nous citons

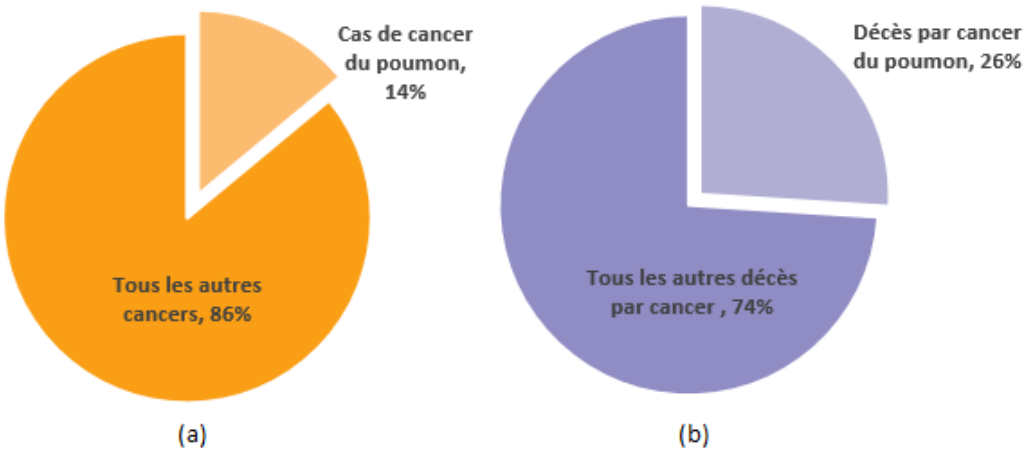


Figure I.2 – (a) : Pourcentage de tous les nouveaux cas de cancer estimés en 2017 chez les deux sexes combinés, (b) : Pourcentage de tous les décès de cancer estimés en 2017 chez les deux sexes combinés

le scanner thoracique. Cet examen présente une technique d'imagerie médicale qui permet de balayer une région de l'organisme et de réaliser des images en coupe à l'aide d'un faisceau de rayons X. Le scanner combine ces rayons à des supports informatiques ce qui permet de reconstruire des images dans des différentes dimensions : on peut ainsi obtenir des vues en coupe longitudinale ou horizontale. Ces images sont connues sous le nom de CT (Computed Tomography) scans. Si le scanner révèle une ou plusieurs anomalies suspectes, il faut alors réaliser une biopsie. Ce geste consiste à récupérer des échantillons de tissus qui semblent anormaux pour les analyser et déterminer s'ils sont de nature cancéreuse ou non[4]. Nous notons que la biopsie est le seul moyen pour déterminer la nature exacte de la tumeur.

2.4 Problématique

Annuellement, les radiologues doivent examiner des milliers d'images. Mais l'interprétation d'un scanner n'est pas toujours facile : «un nodule peut avoir 30 à 40 diagnostics différents», fait valoir le Dr Alain Livartowski, directeur des data de l'ensemble hospitalier de l'Institut Curie (Paris). La figure I.3 présente un ensemble des coupes transversales 2D des tumeurs détectées suite à un scanner thoracique. Nous constatons que le format, la taille et la location de la tumeur ne nous donne pas une interprétation évidente sur sa nature. En plus, une erreur peut conduire à passer à côté d'un cancer ou à infliger une biopsie inutile, ce qui peut s'avérer décisif pour le patient. Ainsi, les tumeurs détectées sont en général dans un stade avancé ce qui décroit les chances de réussite du traitement. En effet, les radiologues se trouvent souvent dans une situation où ils doivent prendre une décision le plus tôt possible tout en conservant une faible probabilité d'erreur. C'est là que les logiciels d'intelligence artificielle entrent en jeu

pour faciliter leur travail, accélérer cette procédure et avoir plus de précision.

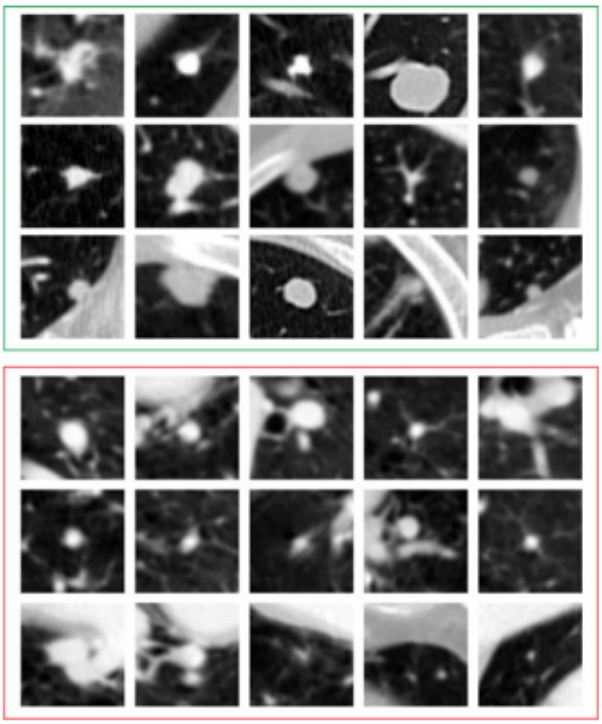


Figure I.3 – Exemples des tumeurs pulmonaires : Le rectangle vert présente des tumeurs malignes et le rouge présente des tumeurs bénignes [1]

3 Le perceptron multi-couches

3.1 Perceptron simple

Le perceptron simple a été introduit en 1958 par Franck Rosenblatt [5]. Il s'agit d'un neurone artificiel inspiré par la théorie cognitive de Friedrich Hayek et celle de Donald Hebb. Dans sa version la plus simple, le perceptron n'a qu'une seule sortie y à laquelle toutes les entrées xi sont connectées. Chaque connexion est pondérée par un poids Wi. Une valeur particulière, appelée biais a également été introduite. Ce biais peut être vu comme une entrée x0 supplémentaire dont la valeur est toujours égale à 1. Celui–ci a notamment été introduit pour un ajustement automatique du seuil, ou encore afin de pouvoir classer facilement un vecteur d'entrée dont toutes les composantes seraient nulles.
La figure I.4 présente un exemple d'un perceptron simple.

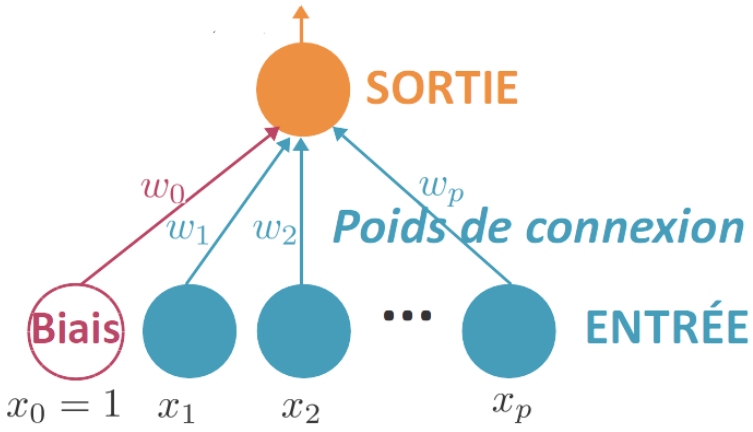


Figure I.4 – Modèle du perceptron simple

3.2 Perceptron multi-couches

Le perceptron multicouche(PMC) est un ensemble de perceptrons répartis dans des couches successives. Le PMC est essentiellement composé de 3 couches[6] : Une couche d'entrée qui accepte les données à traiter, une couche de sortie destinée à délivrer le résultat du calcul et une couche intermédiaire dite couche cachée. Chaque couche est formée d'un ou de plusieurs neurones formels.

3.3 L'apprentissage

L'apprentissage dans un perceptron multicouches est de type supervisé c'est à dire que l'on procède par correction d'erreur (i.e. nous "donnons" au réseau un vecteur d'entrée, le propageons puis calculons l'erreur en sortie par rapport à un vecteur de "sorties désirées" afin de modifier les poids en fonction de cette erreur).
L'apprentissage dans un perceptron peut donc se décomposer en quatre phases :

- La propagation du signal de la couche d'entrée jusqu'à la couche de sortie
- Le calcul de l'erreur en sortie
- La rétro propagation de l'erreur
- La correction des poids

L'apprentissage va donc consister en une minimisation d'une fonction de perte en utilisant l'algorithme de rétro-propagation de l'erreur qui repose sur une minimisation par descente de gradient.

3.4 La descente de gradient

C'est une méthode utilisée souvent pour optimiser les réseaux de neurones, où on doit trouver les paramètres de modèle qui minimisent la fonction de perte sur le jeu de données d'apprentissage. Cela se fait au cours de la phase d'apprentissage, en appliquant une mise à jour sur les paramètres qui par la suite minimise l'erreur de prédiction. Plusieurs algorithmes ont été proposés pour optimiser la descente de gradient.

Batch gradient descent :

C'est la descente de gradient classique. Elle consiste à mettre à jour les poids après parcourir tous les exemples d'entrainement. Cette méthode peut être très lente et irréalisable si les données ne peuvent pas être stockées en mémoire.

La SGD (la descente de gradient stochastique) :

C'est une méthode qui met à jour les paramètres pour chaque exemple x de l'ensemble de données associé au label y. Cependant, cette méthode est trop coûteuse car elle nécessite d'évaluer le modèle pour chaque exemple de la base de données.

Mini-batch gradient descent :

Cette méthode consiste à mettre à jour les poids après l'exécution de chaque mini groupe de n exemples d'apprentissage :Généralement les mini groupes contiennent de 50 à 256 exemples[7].

Momentum :

La méthode du momentum (Polyak, 1964) est conçue pour accélérer le processus de descente de gradient. Ceci se fait par l'introduction d'une variable qui joue le rôle d'un vecteur de vélocité. C'est la direction et la vitesse à laquelle les paramètres se déplacent dans l'espace paramétrique. On définit un hyper-paramètre lambda1 permettant d'amortir la vélocité quand on atteint une surface plate.

RMSprop :

RMSprop est une méthode proposée par Geoff Hinton dans Lecture 6e de sa Coursera Classe[8].L'algorithme ajuste automatiquement le taux d'apprentissage à chaque paramètre. Il ne cumule que les gradients issus des itérations récentes. Pour cela, il utilise une moyenne glissante.

Adam :

ADAM (Adaptive Moments) est l'un des algorithmes les plus récents et les plus efficaces pour l'optimisation de descente de gradient. Il permet d'adapter automatiquement le taux d'apprentissage pour chaque paramètre. Pour la mise à jour des paramètres, il combine les deux algorithmes cités précédemment « Momentus » et « RMSprop ».

3.5 Métriques

Plusieurs métriques ont été proposées pour mesurer la performance d'un modèle donné. Dans ce projet, nous nous intéressons essentiellement à la matrice de confusion et l'AUC.

Matrice de confusion

Prenons l'exemple d'un classifier binaire. Pour mesurer les performances d'un réseau, nous distinguons 4cas de figures :

- VP (Vrai Positif) : Exemple positif correctement classé
- VN (Vrai Négatif) : Exemple négatif correctement classé
- FP (Faux Positif) : Exemple positif mal classé
- FN (Faux Négatif) : Exemple négatif mal classé

Toutes ces informations peuvent être regroupés et rassemblés sous forme d'un tableau illustré ci-dessous, et nommé «Matrice de confusion » :

		Classe prédite	
		No	Yes
Classe réelle	No	VN	FN
	YES	FP	VP

En particulier, si la matrice de confusion est diagonale, c'est-à-dire les FP et les FN sont nuls, le classifier est considéré parfait.

Courbe ROC

La courbe ROC est une représentation du taux de vrais positifs en fonction du taux de faux positifs. Son intérêt est de s'affranchir de la taille des données de test dans le cas où les données sont déséquilibrées. Cette représentation met en avant un nouvel indicateur, appelé AUC (Area Under Curve) qui est l'aire sous la courbe. Cette valeur mesure l'intégralité de l'aire à deux dimensions située sous l'ensemble de la courbe ROC de (0,0) à (1,1). Plus elle se rapproche de 1, plus le classifieur est performant.

Contrairement au matrice de confusion, l'AUC est indépendante des seuils de classification. Elle mesure la qualité de précision du modèle quel que soit le seuil fixé.

4 Deep Learning

A présent que nous avons précisé comment fonctionnent les réseaux de neurones de manière générale, nous allons aborder le domaine du Deep Learning.

Le Deep Learning (ou apprentissage profond) est une forme d'intelligence artificielle, dérivée

de Machine Learning (apprentissage automatique). Ce n'est que récemment, grâce à l'avancée des performances de calcul des ordinateurs, que ce concept s'est développé.
Cette famille d'algorithmes a permis de faire des progrès importants dans des domaines différents tels que l'analyse du signal sonore et visuel, la reconnaissance faciale et vocale, la vision par ordinateur et notamment la classification des images. Cet apprentissage tente de modéliser avec un haut niveau d'abstraction des données grâce à des architectures articulées de différentes transformations non linéaires. En effet, le modèle de deep learning a la capacité d'extraire des caractéristiques à partir des données brutes en traversant les multiples couches cachées[9].

5 Les réseaux de neurones convolutifs

Dans notre projet nous nous intéressons particulièrement au réseau de neurones convolutifs (CNN) qui constitue une sous-catégorie de réseaux de neurones profonds.

5.1 Définition

Les réseaux de neurones convolutifs, appelés aussi CNN ou ConvNet sont des réseaux qui utilisent la convolution à la place de la multiplication matricielle dans au moins une de leurs couches. Ce type de réseau de neurones est spécialisé pour le traitement de données ayant une topologie semblable à une grille [10].

5.2 Architecture générale

Pour la classification des images, ces réseaux comportent deux parties bien distinctes.
La première partie d'un CNN est la partie convolutive à proprement parler. Elle fonctionne comme un extracteur de caractéristiques des images. Une image est passée à travers une succession de filtres, créant de nouvelles images appelées cartes de convolutions. Certains filtres intermédiaires réduisent la résolution de l'image par une opération de maximum local. Au final, les cartes de convolutions sont mises à plat et concaténées en un vecteur de caractéristiques, appelé code CNN.
Ce code CNN en sortie de la partie convolutive est ensuite branché en entrée d'une deuxième partie, constituée de couches entièrement connectées (perceptron multicouche). Le rôle de cette partie est de combiner les caractéristiques du code CNN pour classer l'image. La sortie est une dernière couche comportant un neurone par catégorie[11].

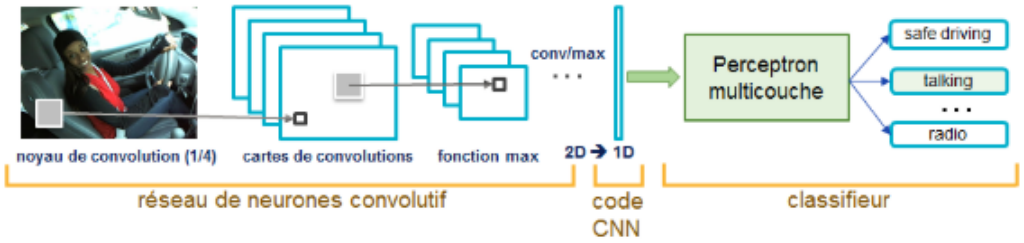


Figure I.5 – Architecture du CNN

5.3 Les couches

Il existe quatre types de couches pour un réseau de neurones convolutif : la couche de convolution, la couche de pooling, la couche d'activation et la couche fully-connected (ou couche entièrement connectée) :

La couche de convolution : La convolution est le fait de mettre en valeur quelques caractéristiques bien choisies dans l'image source. Cette opération est faite en appliquant une matrice prédéfinie sur la matrice source de l'image initiale. C'est la composante principale des réseaux de neurones convolutifs qui doit constituer toujours au moins leur première couche.

La couche de Pooling : Le pooling est une forme de sous-échantillonnage. Il réduit la taille spatiale d'une image tout en conservant ses caractéristiques pertinentes. Ainsi, il réduit la quantité des paramètres et de calcul dans le réseau. La forme la plus courante est une couche de mise en commun avec des matrices de taille 2x2 et comme valeur de sortie la valeur maximale en entrée. On parle dans ce cas de « Max-Pool 2x2 »[7].

Il est possible d'utiliser d'autres fonctions de pooling que le maximum dont on cite « average pooling» où la sortie est la moyenne des valeurs du patch d'entrée. Cependant, le max-pooling est plus efficace que le « average pooling » car celui-ci augmente plus significativement l'importance des activations fortes.

Couches de correction : Il est possible d'améliorer l'efficacité du traitement en intercalant entre les couches de traitement une couche qui va opérer une fonction d'activation sur les signaux de sortie.

Couche entièrement connectée (Fully connected) : Après plusieurs couches de convolution et de max-pooling, le raisonnement de haut niveau dans le réseau neuronal se fait via l'implémentation d'une ou de plusieurs couches entièrement connectées qui permettent la classification de l'image en se basant sur les caractéristiques détectées précédemment grâce aux couches de convolution. Les neurones dans une couche entièrement connectée ont des connexions vers toutes les sorties de la couche précédente. En général, la dernière couche FC est suivie par une fonction d'activation soft max présentant une probabilité entre 0 et 1 pour chaque catégorie

possible[12].

Couche de normalisation (Batch normalization) : L'apprentissage du réseau converge plus rapidement si ses entrées sont centrées réduites c'est-à-dire que la moyenne est égale à 0 et la variance est égale à 1. Comme chaque couche observe des entrées produites par l'ensemble des couches qui la précèdent, il serait avantageux d'obtenir des entrées centrées et réduites pour chaque couche. La couche de normalisation est une composante introduite entre les couches du réseau de neurones et qui prend continuellement la sortie d'une couche particulière et la normalise avant de l'envoyer à la couche suivante comme entrée.

5.4 Avantages

Les CNN visent à limiter le nombre d'entrées tout en conservant la forte corrélation « spatialement locale » des images naturelles. Pour ce faire, la convolution repose sur trois idées principales :

1. **Volumes 3D de neurones :** La couche de neurones n'est plus simplement une surface (perceptron), mais devient un volume avec une profondeur. Si on considère un seul champ récepteur du CNN, les n neurones associés (sur la profondeur) forment l'équivalent de la première couche d'un PMC[7].

2. **Connectivité locale :** Les réseaux de neurones traditionnelles se basent sur la multiplication matricielle en utilisant une matrice de paramètres avec des paramètres séparés décrivant la relation entre chaque unité d'entrée et chaque unité de sortie. Cela signifie que chaque unité de sortie interagit avec chaque unité d'entrée ce qui n'est pas le cas pour les réseaux de neurones convolutifs. Ceci est accompli en rendant le noyau plus petit que l'entrée.
   Par exemple, lors du traitement d'une image, l'image d'entrée peut avoir des milliers ou des millions de pixels, mais nous pouvons détecter de petites caractéristiques significatives telles que les bords avec des noyaux qui n'occupent que des dizaines ou des centaines de pixels. Cela signifie que nous pouvons stocker moins de paramètres, ce qui réduit les besoins en matière de mémoire du modèle et améliore son efficacité. Cela signifie également que le calcul de sortie nécessite moins d'opérations[7].
3. **Paramètres partagés :** Le partage des paramètres se réfère à l'utilisation du même paramètre pour plus d'une fonction dans un modèle. Dans un réseau de neurones convolutif, chaque élément du noyau est utilisé à chaque position de l'entrée (sauf peut-être quelques-

uns des pixels des bords, selon le choix de conception concernant la frontière). Le paramètre sharing utilisé par l'opération de convolution signifie que, plutôt que d'apprendre un ensemble de paramètres distincts pour chaque emplacement, nous n'apprenons qu'un ensemble ce qui réduit encore davantage les exigences de stockage du modèle[7].

5.5 Les paramètres

Avant de commencer l'entrainement, il faut fixer les paramètres de chaque couche :

- Le taux d'apprentissage ou le 'Learning rate' : utilisé lors de la mise à jour des poids. Il peut être accompagné par un terme de pénalisation appelé decay.
- Profondeur de la couche : nombre de filtres utilisés, noté K. La profondeur détermine le nombre de caractéristiques détectées.
- Le pas (Stride) : contrôle le chevauchement des champs, noté s.
- La marge (zero padding) : parfois, il est recommandé de mettre des zéros à la frontière du volume d'entrée. La taille de ce 'zero-padding' est le troisième paramètre, noté p. Cette marge permet de contrôler la dimension spatiale du volume de sortie. En particulier, il est parfois souhaitable de conserver la même surface que celle du volume d'entrée.
- La taille de filtre : noté F*F

6 Présentation du projet

6.1 Classification des tumeurs

La complexité du processus de détection du cancer du poumon, les expertises demandées lors du diagnostic, et la contrainte du temps créent une nécessité d'implémenter un CAD qui permet de classifier les tumeurs pulmonaires. Notre projet vient apporter une solution à ces contraintes par un gain de précision, de rapidité et surtout par une réduction allant de 14 à 20% du taux de décès causés par ce type du cancer[13].
La solution proposée se matérialise par la création d'un algorithme de deep learning basé sur les réseaux de neurones convolutifs 3D. Nous voulons développer une méthode automatisée qui détermine à partir des images médicales de CT scan si les lésions pulmonaires sont cancéreuses en utilisant l'ensemble de données disponibles.
Grace à un tel système, non seulement nous pouvons mieux diagnostiquer une personne atteinte de cancer, mais nous pouvons également affirmer si une personne n'a pas le cancer, ce qui

pourrait lui épargner une biopsie pulmonaire invasive, coûteuse et risquée.

6.2 Planification du projet

Nous présentons dans la figure I.6 la réparation des phases de notre projet sous forme d'un diagramme de Gant :

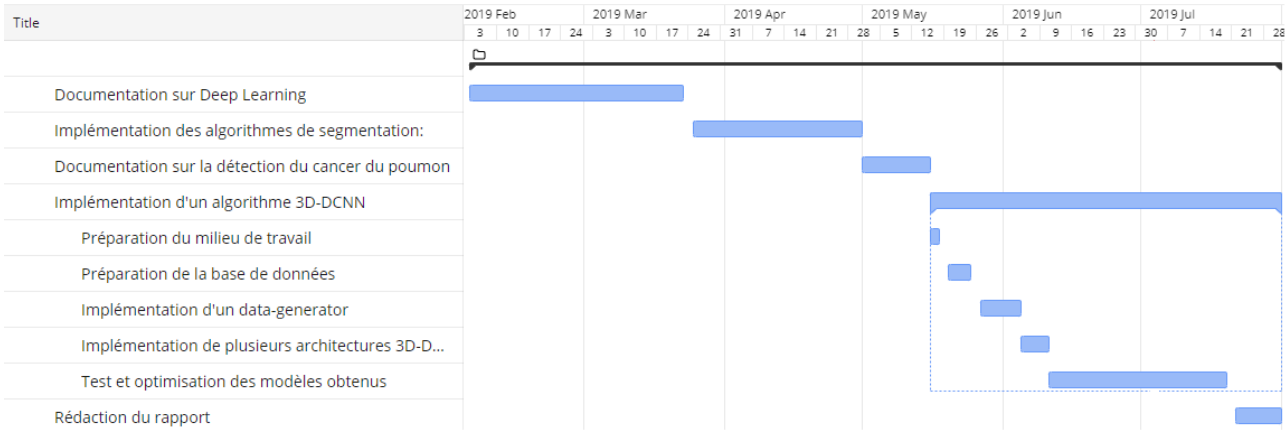


Figure I.6 – Diagramme de Gantt

Conclusion

Dans ce premier chapitre, nous avons présenté l'organisme d'accueil, ainsi que le cadre général de ce projet tout en définissant les notions principales qui vont nous servir comme des prérequis pour les chapitres suivants.

Chapitre II

Etude de l'existant

Introduction

La détection de cancer du poumon introduit essentiellement deux phases : La première phase est la segmentation qui consiste à extraire à partir de chaque image de CT scan l'information pertinente présentant dans notre cas la tumeur. De nombreuses techniques ont été trouvées, certaines plus performantes que d'autres, mais la plus connue est UNet. Ce type de réseau est composé d'une partie contractante et une voie expansive, ce qui confère une architecture en forme de « U ». La deuxième phase est la classification qui consiste à attribuer de façon automatique un objet à une classe ou une catégorie, parmi d'autres classes possibles. Un algorithme qui réalise automatiquement une classification est appelé un classifier.
Dans ce projet, nous allons se focaliser sur la partie de classification puisque la base de données sur laquelle nous allons travailler provoque des annotations indiquant la position en x, y et z de chaque tumeur, d'où nous allons ignorer la phase de segmentation. Dans ce chapitre, nous allons aborder quelques approches utilisées pour la classification des tumeurs.

1 Les approches 2D

Principalement, deux approches 2D ont été proposées pour appliquer la classification des images CT scans.

1.1 CNN/ELM

Dans cette approche [14], le modèle est constitué de deux algorithmes : Le CNN et l'ELM (Extreme Learning Machine).
Pour avoir des bons résultats de classification, le nombre des neurones recommandés est 1000 neurones ou plus. Cependant, l'augmentation de nombre de neurones engendre une augmentation de complexité du modèle et affecte sa rapidité d'entrainement, d'où l'introduction d'une telle approche. En effet, cette approche permet l'extraction des caractéristiques les plus robustes d'une tumeur pulmonaire en utilisant l'auto encodeur MSAE qui réfère à un ensemble de bloc SAE. Le SAE [15] est un algorithme non supervisé, constitué d'une architecture de réseau de neurones spécifique qui vise à entrainer le modèle de manière à représenter les données en entrée via la retro propagation. Il présente deux parties principales : un encodeur qui transforme l'entrée X à une représentation h et un décodeur qui sert à reconstruire une approximation de X à partir de h toute en minimisant le plus possible l'erreur entre les deux. La figure II.1 illustre une architecture de SAE :

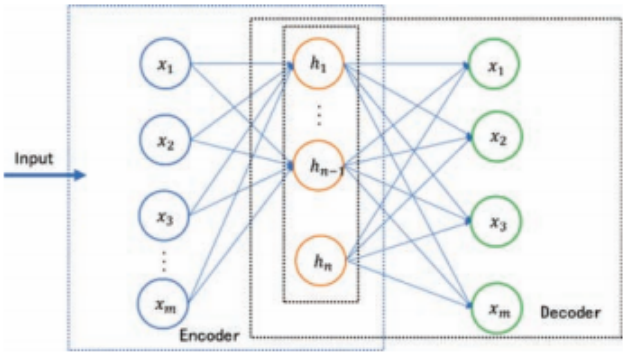


Figure II.1 – Architecture de SAE

L'introduction de cet algorithme a pour but de diminuer le nombre de pixels présenté dans une image, produire une meilleure carte de caractéristiques et ne conserver que l'information pertinente utilisée par la suite dans l'entrainement. Cela se fait par la détermination de la corrélation entre les pixels.
L'exécution de l'algorithme avec des formats différents se fait avec le MSAE. Il s'agit de deux blocs SAE comme illustré dans la figure II.2. W1 et W2 présentent les deux matrices de poids entre les couches. La couche d'entrée est représentée par 1024 pixels de tranches de format 32*32. La 2eme couche cachée est constituée de 16 neurones avec un hn égal respectivement à 25,100 et 169 neurones.
La sortie de MSAE permet l'obtention de plusieurs classifier, d'où l'introduction de l'algorithme

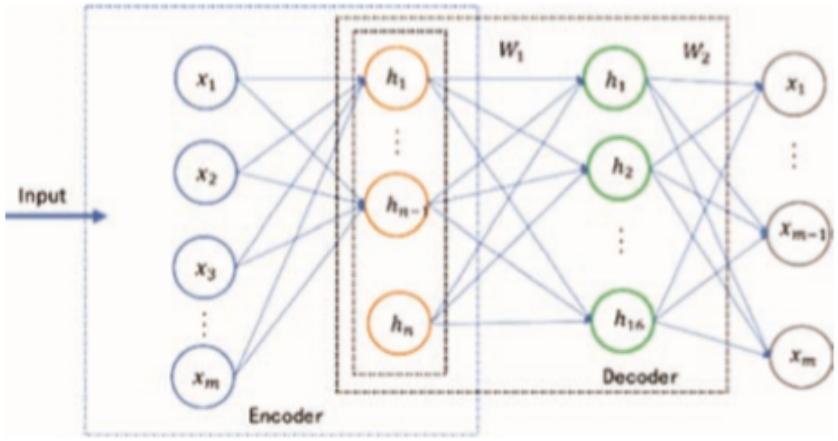


Figure II.2 – Architecture de MSAE

ELM. Ce dernier sert à combiner ces classifiers tout en provoquant une rapidité d'entrainement et des performances exceptionnelles. Finalement, ils seront convolués et passés par un SVM (Support Vector Machine) pour trouver l'hyperplan le plus convenable.
L'étude de cet algorithme a été faite en utilisant 10663 tranches du format 32*32, extraites à partir de 283 images CT scans de LIDC. Les nodules sélectionnés ont un diamètre supérieur à 3mm.
Finalement, l'algorithme permet d'avoir 96% d'exactitude pour la classification des tumeurs malignes. En comparant celui-ci avec un réseau présentant un seul bloc de SAE, cet algorithme peut extraire des caractéristiques différentes de chaque niveau, et par la suite améliorer les performances de classification. Cependant, comme cité précédemment, le scan provoque des images en 3D, donc seule la tranche qui présente la plus grande surface de tumeur sera considérée. Dans ce cas, nous notons une perte des informations situées dans d'autres tranches.

1.2 Multi-view CNN

La méthode par laquelle nous déterminons les images candidates joue un rôle très important dans l'amélioration des performances d'un système CAD puisqu'elle devrait identifier idéalement toutes les lésions suspectes. Cependant, la variation morphologique des nodules est souvent supérieure à ce qu'un algorithme de détection des candidats peut détecter.
Dans cette approche [16], au lieu de prendre de chaque image 3D une seule tranche, plusieurs seront extraites en référant à des plans fixés auparavant.
L'architecture de cette approche est illustrée dans la figure II.3.

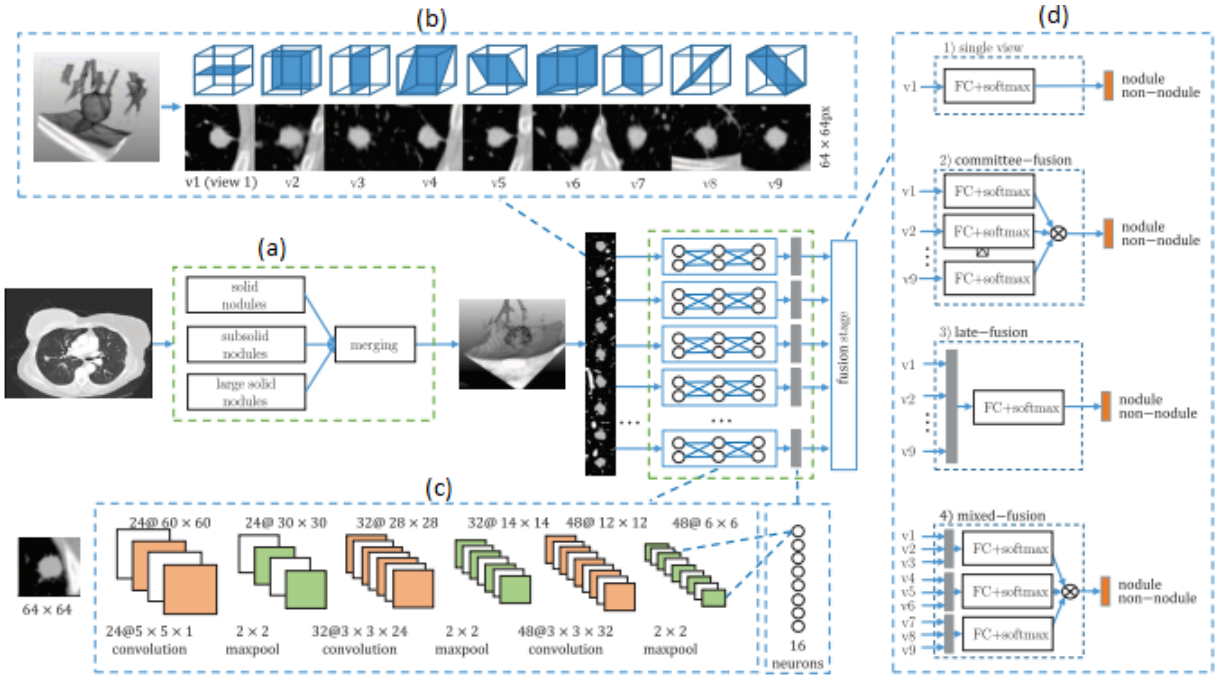


Figure II.3 – Architecture de l'approche Multi-view CNN

1.2.1 Extraction des images candidates

Pour détecter un plus large spectre des tumeurs, ils ont appliqué une combinaison de plusieurs algorithmes utilisés pour la détection de candidats [17][18][19]. Trois algorithmes existants sont proposés ou chacun vise un type spécifique de nodules, à savoir nodules solides, sub-solides en verre dépoli pur (VDP) ou largement solides.

Pour les nodules solides, définis par une densité tissulaire effaçant les structures parenchymateuses sous-jacentes, ils ont mis en œuvre la technique proposée par Murphy et al [17]. Pour chaque voxel, le format et la courbure sont calculées et un seuillage est appliqué sur les deux mesures pour définir les points de départ. Une méthode de segmentation automatique est exécutée aux points de départ pour obtenir des clusters d'intérêt. Les clusters les plus proches sont fusionnés. Finalement, les clusters ayant un volume <40 mm3 sont éliminés.

Pour les VDP, caractérisés par une densité suffisamment faible pour maintenir visibles les structures vasculaires ou bronchiques sous-jacentes, ils ont mis en œuvre la technique proposée par Jacobs et al[18]. Un masque à double seuil est d'abord effectué pour obtenir un masque avec des voxels d'intérêt. Une ouverture morphologique est appliqué pour supprimer les clusters connectés, suivi de l'analyse 3D des composants connectés. Enfin, une segmentation des candidats est obtenue en utilisant un algorithme de segmentation déjà existant [20].

Les nodules largement solides (>= 10mm), ayant des surfaces différentes de celles des petites

lésions, ont une plage d'intensité spécifique et ils ne peuvent pas être capturée par les algorithmes de détection des nodules solides ou sub-solides. Par conséquent, les deux algorithmes susmentionnés ne fonctionnent pas bien dans la détection de ce type de nodules. Un troisième algorithme [19] composé de trois étapes a été proposé :

— Appliquer un post-traitement de la segmentation
— Appliquer seuillage de densité, pour obtenir les voxels d'intérêt
— Appliquer l'ouverture morphologique dans plusieurs étapes pour obtenir des groupes de candidats

La sortie de chaque algorithmes est sous la forme de 2 objets présentant la position p de chaque tumeur, ainsi que la probabilité pour qu'elle soit cancéreuse.
Finalement, trois séries de nodules candidates sont calculées et fusionnées. La position et la probabilité de chaque nodule sont moyennées.

1.2.2 Extraction des tranches

Pour chaque image candidate, l'algorithme extrait un multiple des tranches en 2D de 50*50 mm centrées en p. La taille des tranches est déterminée de façon qu'elle présente d'une manière complète le nodule et qu'elle contient l'information pertinente pour pouvoir la classifier.
Neuf tranches sont extraites par rapport les plans de symétrie d'un cube comme présenté dans la figure II.3 à l'étape(a), et ses résolutions sont ensuite converties vers 64*64 pixels.

1.2.3 Configuration de réseau 2D-CNN

Le réseau CNN est construit en combinant différents flux de ConvNets, appelés multi-view. Chaque vue traite une tranche d'image 2D à partir une vue(un plan de symétrie) spécifique de l'mage candidate.
L'architecture de 2D-CNN, présentée dans la figure II.3, est inspirée d'une autre étude où tous les hyper paramètres sont configurés et fixés sur les valeurs optimales. C'est pour cela qu'ils identifient deux autres paramètres critiques à modifier afin de les optimiser. Ces deux paramètres sont le nombre des vues et la méthode de fusion. Ce réseau est composé de 3 couches de convolution, chacune est suivie par une couche MaxPooling. La dernière couche FC est composée de 16 neurones et la fonction d'activation utilisée est ReLU.

1.2.4 Fusion des flux ConvNets

Trois méthodes de fusion des flux ConvNets ont été proposées :
Committee-Fusion : Cette méthode [21] consiste à diviser la tâche de détection d'objet 3D en

plusieurs tâches simples de détection en 2D. La sortie de la dernière couche FC de chaque flux est connectée à une couche de classification présentant une couche supplémentaire entièrement connectée avec fonction d'activation softmax. Le résultat final se présente par la multiplication des probabilités de chaque flux comme illustré dans la figure II.3 en (d).

Late-Fusion : La méthode de fusion tardive [22] concatène les sorties des premières couches FC et injecte le résultant directement dans la couche de classification (voir figure II.3 (d)). Dans cette configuration, les paramètres des couches de convolution de différents flux sont partagés.

Mixed-Fusion : La fusion mixte [23] est une combinaison des deux approches précédentes. Profitant des multiples vues, la prévision du système est améliorée en combinant plusieurs sorties de late-fusion dans un seul bloc de commitee-fusion. Cela se fait par la division des neufs flux en trois ensembles indépendants.

Plusieurs bases de données ont été utilisées pour cette étude dont nous citons LIDC-IDRI (The Lung Image Database Consortium and Image Database Resource Initiative) constituée de 1018 cas collectés de sept institutions. ANODE09 constituée de 55 images de CT scans collectées du centre médical de l'université Utrecht et une autre base de données provoquée par le DLCST (Danish Lung Cancer Screening Trial).

Finalement, le modèle a atteint la meilleure valeur de métrique avec 99,6% de AUC en utilisant la méthode mixed-fusion et en fixant le nombre de vues à neuf. Ils ont ainsi noté, que l'ajout d'autres vues peut améliorer de plus les performances du modèle.

2 Les approches 3D

Multi-level Contexual CNN

Cette approche [1] consiste à utiliser trois réseaux 3D-CNN en parallèle, tout en modifiant à chaque fois le format de tumeur en entrée afin de détecter la variation ces caractéristiques telles que la taille, la forme, et les informations situées autour de tumeur. Ainsi, ce modèle utilise les noyaux en 3D, ce qui permet l'exploitation totale de l'information provoqué par les tumeurs d'entrainement. Finalement, les trois sorties sont combinées, d'où la mise en place de la classification.

La figure II.4 présente l'architecture de cette approche.

Les trois réseaux sont constitués chacun de trois couches de convolution 3D, une couche de MaxPooling après la première couche de convolution, et deux couches FC après la dernière couche de convolution, suivie par une couche d'activation softmax pour estimer la probabilité des tumeurs. Toutes les couches utilisent la fonction d'activation ReLu. Tous les autres détails sont présentés dans le tableau II.1 :

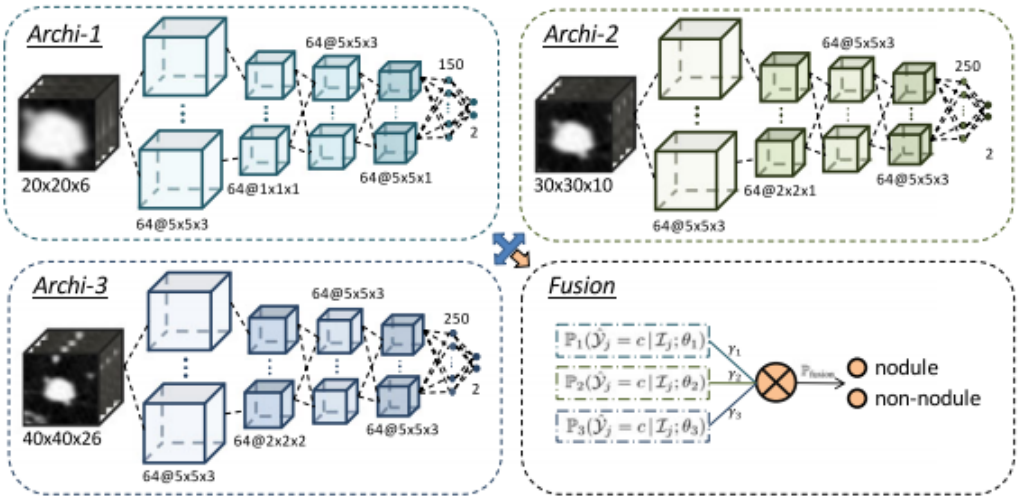


Figure II.4 – Architecture de l'approche multi-level contexual CNN

Arch1			Arch2			Arch3		
Layer	**Kernel**	**Channel**	**Layer**	**Kernel**	**Channel**	**Layer**	**Kernel**	**Channel**
Input	**-**	**1**	**Input**	**-**	**1**	**Input**	**-**	**1**
C1	**5*5*3**	**64**	**C1**	**5*5*3**	**64**	**C1**	**5*5*3**	**64**
M1	**1*1*1**	**64**	**M1**	**2*2*1**	**64**	**M1**	**2*2*2**	**64**
C2	**5*5*3**	**64**	**C2**	**5*5*3**	**64**	**C2**	**5*5*3**	**64**
C3	**5*5*1**	**64**	**C3**	**5*5*3**	**64**	**C3**	**5*5*3**	**64**
FC1	**-**	**150**	**FC1**	**-**	**250**	**FC1**	**-**	**250**
FC2	**-**	**2**	**FC2**	**-**	**2**	**FC2**	**-**	**2**
Softmax	**-**	**2**	**Softmax**	**-**	**2**	**Softmax**	**-**	**2**

Tableau II.1 – Architecture de l'approche Multi-contextual 3D-CNN

Le modèle a été configuré de la manière suivante : La mise à jour des poids se fait à l'aide de SGD. Les cubes centrées en p sont de la taille 20 × 20 × 6, 30 × 30 × 10, et 40 × 40 × 26 correspondant aux trois architectures. Pour régler l'effet de déséquilibrage entre les classes, plusieurs techniques d'augmentation de données sont appliquées telles que a translation et la rotation. En total, 0.65 million d'exemples d'entrainent ont été obtenu. Pendant la phase d'entrainement, les poids sont initialisés aléatoirement suivant la distribution gaussienne N (0, 0.012). Le taux d'apprentissage a été fixé à 0.3 avec un decay de 5% chaque 5000 itérations. La taille de mini batch a été fixée à 200 et le momentum à 0.9.
Pour évaluer le modèle, la base de données LUNA16 a été utilisée. En appliquant la validation croisée de 10 fois, un CPM (Competition Performance Metric) de 0,827 et une sensibilité de 0,922 à 8 FP / scan ont été atteints.

Conclusion

Les images CT scan ont prouvées leurs performances dans le diagnostic de cancer du poumon. Mais la nature de ces dernières, présentant la forme 3D, met les approches 2D vis-à-vis un challenge. Dans les chapitres suivants, nous allons aborder une solution de classification de cancer du poumon en 3D.

Chapitre III

Optimisation du calcul et du traitement

Introduction

Dans un réseau de neurones, deux modes de fonctionnement peuvent être distingués. Dans le premier, les paramètres du réseau sont ajustés par l'algorithme de retro-propagation grâce à la présentation d'exemples pour lesquels on connaît la réponse désirée : ce mode de fonctionnement est appelé « phase d'apprentissage ». Dans le deuxième mode, il s'agit d'exploiter le réseau en lui présentant des nouvelles données non étiquetées : ce mode d'utilisation est souvent appelé « phase de test», ou « phase de généralisation ».
Dans ce chapitre, nous allons aborder les différents cas de figure à la fin de la phase d'entraînement et les solutions possibles pour remédier aux problèmes rencontrés.

1 Résultats possibles

A la fin du processus d'apprentissage, trois cas de figure [24] peuvent se présenter :

— Le modèle est aussi performant sur les données d'entraînement que sur les données de validation. Cela est le cas de figure idéale, ça signifie que le modèle a très bien fait son travail.

— Le modèle reconnaît très bien les images d'entraînement et moins bien celles de validation. Le modèle aura une faible capacité prédictive, il n'arrive pas à généraliser. Ce cas se manifeste par un taux d'erreur élevé dans la phase de validation. On parle alors des données surexploitées ou de sur-apprentissage (overfitting).

— Le modèle ne reconnaît pas très bien les images d'entraînements et pas très bien non plus les images de validation. On parle alors de sous-apprentissage (underfitting).

La figure III.1 présente les différents cas de figure :

Dans le cas de sous-apprentissage, il faudrait utiliser un modèle plus complexe en appliquant

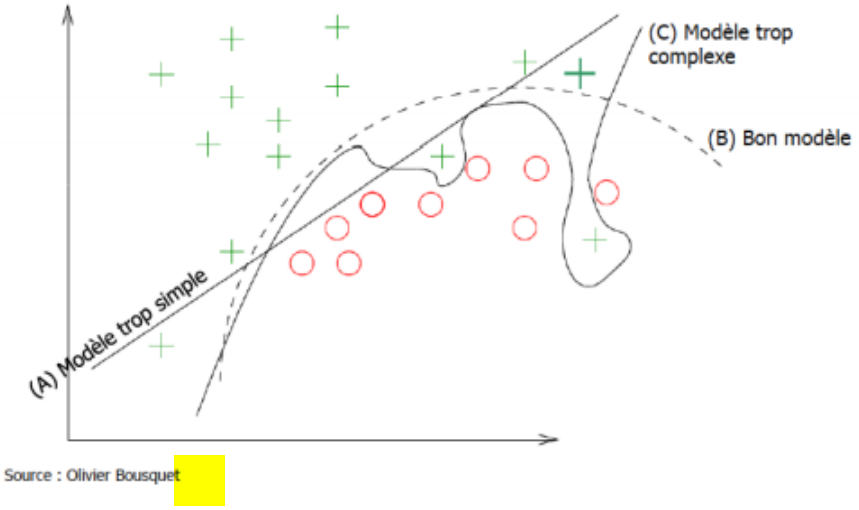


Figure III.1 – Les cas de figure possible lors de la phase d'apprentissage

une ou plusieurs de ces méthodes :

— Ajouter d'autres couches cachées pour rendre le réseau plus profond.
— Entraîner le modèle de plus
— Essayer une autre architecture de réseau.

Dans le cas de surexploitation des données, d'autres méthodes ont été développées afin de généraliser le modèle et par la suite réduire ce phénomène. Parmi les solutions qui répondent à ce problème, on trouve :

— Ajouter plus de données d'entrainement.
— Essayer une autre architecture de réseau.
— Appliquer une ou plusieurs méthodes de régularisation

2 La régularisation

De nombreuses stratégies utilisées dans domaine de Deep Learning sont explicitement conçues pour réduire l'erreur de test. Ces stratégies sont connues sous le nom de régularisation.

2.1 L2-régularisation (Weight decay)

Cette régularisation se manifeste par l'ajout d'une pénalité appelée facteur de régularisation à la fonction de perte. Au lieu de seulement chercher à minimiser la perte, nous allons minimiser à la fois la perte et la complexité du modèle de telle sorte que l'algorithme devient une fonction à deux facteurs : le facteur de perte, qui mesure l'efficacité d'apprentissage du modèle, et le facteur de régularisation, qui mesure sa complexité.
On peut quantifier la complexité à l'aide de la formule de L2, qui définit le facteur de régularisation comme étant la somme des carrés de tous les poids Wi :
L2 :Facteur de régularisation= $\frac{1}{2}\ \|W\|_F^2$
Les développeurs de modèles ajustent l'effet global de ce facteur en multipliant sa valeur par une grandeur scalaire appelée lambda (ou taux de régularisation) :
J(w)=$\frac{1}{M}\sum_{i=1}^{i=M} L(\tilde{y_i}, y_i) + \frac{\lambda}{2M}\sum_{i=1}^{i=M} W_i$ Où M est le nombre d'exemples d'entrainement [25]
Comme illustré dans la figure III.2, l'augmentation du lambda renforce les effets de la régularisation : Plus qu'on augmente lambda, plus que les valeurs des poids deviennent plus faibles.

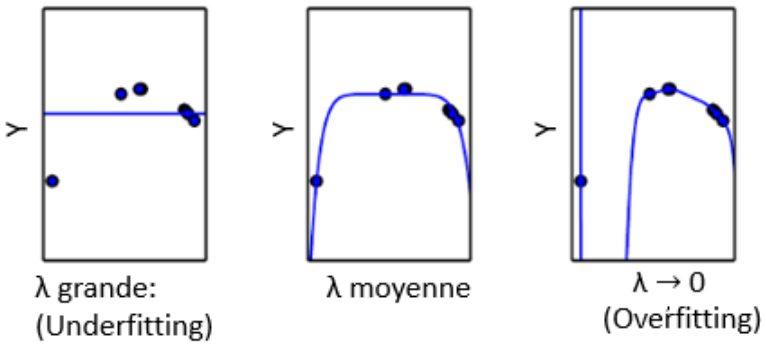


Figure III.2 – L'effet de lambda sur l'apprentissage

2.2 Dropout

Le Dropout consiste à entrainer chaque exemple avec un réseau plus petit. Il permet à la fois de réduire le sur-apprentissage et de combiner plusieurs architectures de réseau. Le terme dropout se réfère à la mise en veille temporairement d'un certain pourcentage de neurones en chaque itération et par la suite l'élimination de toutes les connexions sortant de ces neurones. Cela signifie que leur contribution à l'activation des neurones qui leur succède est temporairement supprimé lors de la phase de propagation et toutes les mises à jour de poids ne sont pas appliquées au neurone lors de la phase de retro propagation Cela se fait selon une probabilité

p fixée pour chaque couche. Le dropout peut être interprété comme une méthode de régularisation du réseau de neurones qui se manifeste par l'ajout du bruit aux couches cachées. Lors de la phase de test, tous les neurones sont gardés [26].

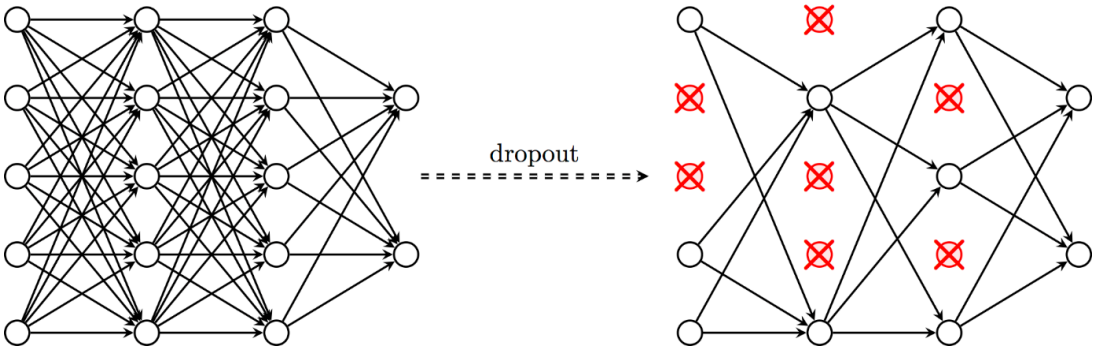


Figure III.3 – Dropout

2.3 Early stopping

C'est la méthode de régularisation la plus utilisée. Elle consiste à calculer le taux d'erreur périodiquement durant l'entrainement et arrêter l'apprentissage quand ce taux augmente de nouveau. En effet, normalement, le taux d'erreur d'entrainement et de validation diminue durant la phase d'apprentissage. Cependant, quand le réseau commence à surexploiter les données, l'erreur de validation commence à augmenter [27].
La figure III.4 [28] illustre la méthode de Early Stopping.

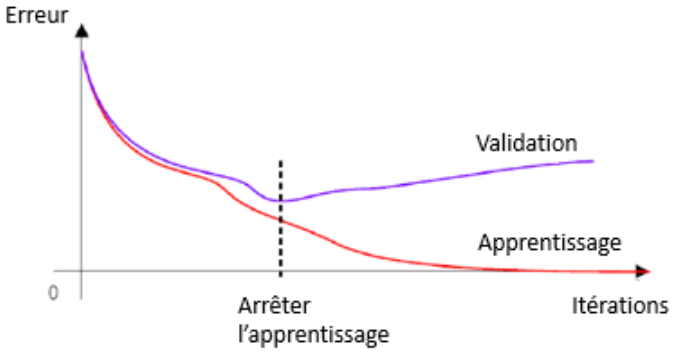


Figure III.4 – Early Stopping

2.4 Augmentation des données

2.4.1 Phase de prétraitement

De manière générale, en data science, les données n'arrivent jamais sous une forme directement exploitable par le data scientist. Le Deep Learning ne fait pas exception. En réalité, le plus gros travail consiste à appliquer le prétraitement convenable sur les données : forme, composition etc... En effet, lors de la phase d'entrainement le réseau ne saura pas vers quelles valeurs se tourner pour apprendre des caractéristiques de cette image, et lors de la phase de prédiction, nous n'aurons aucun moyen de vérifier rigoureusement ce que fait le réseau dessus. Ceci implique donc de faire un filtrage des données.

2.4.2 Analyse des données

Pour préciser le prétraitement nécessaire, il est toujours conseillé de visualiser les données sur lesquelles on souhaite travailler par la suite. Cela permettra de faire de meilleurs choix sur la nature et les caractéristiques de la base de données que l'on présente au réseau.

2.4.3 Equilibrage de la base de données

Suite à la phase d'analyse des données, le problème majeur qui peut se présenter est le déséquilibrage des exemples positifs et négatifs. Sachant que parfois il est difficile de collecter les images médicales et que cette procédure peut être très chère, donc la solution consiste à appliquer des méthodes permettant l'augmentation des données.

2.4.4 Augmentation des données

Il existe plusieurs techniques qui permettent d'augmenter le nombre des exemples. Parmi eux, nous citons la translation de l'image, le cropping qui permet de zoomer l'image sur des endroits, la rotation, le changement d'intensité, l'effet miroir (Flipping) etc. Notez bien que ces méthodes permettent non seulement le balancement des exemples d'entrainement, mais aussi elles présentent une sorte de régularisation, permettant la réduction de l'effet de sur-apprentissage.

3 Optimisation des performances de calcul

3.1 Le GPU

Le GPU (Graphics Processing Unit) est l'outil indispensable pour faire du Deep Learning. Lancés en 2007 par NVIDIA pour des raisons graphiques, les GPU accélèrent les portions de code lourdes en ressources de calculs, les portions restantes étant traitées par le CPU (Center Processing Unit). Les calculs parallélisés des réseaux de neurones sont distribués sur les milliers cœurs GPU et accélèrent le temps d'entraînement.

L'architecture matérielle conseillée pour entraîner un réseau de neurones convolutif est une

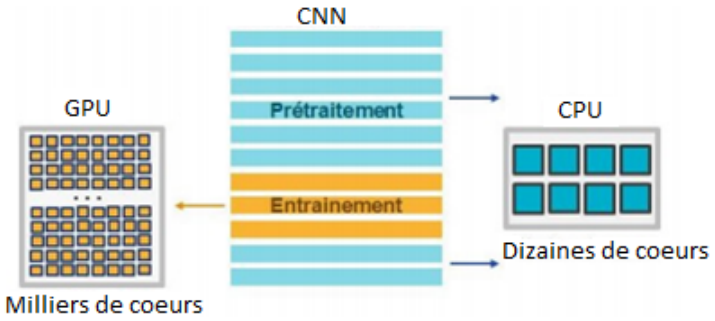


Figure III.5 – Répartition des calculs entre le CPU et le GPU

carte graphique. Il est possible d'utiliser les CPUs mais le temps d'entraînement à de forte chance d'exploser, passant de 20h avec GPU à 200h sur CPU. Comme indiqué précédemment, l'assise du groupe NVIDIA ore à ses cartes graphiques une utilisation performante et documentée. Il est également possible d'utiliser le cloud d'Amazon Web Service pour lancer ses calculs à distance.

3.1.1 Entrainer le réseau sur plusieurs GPUs sur une seule machine

L'un des librairies de python qui supporte le parallélisme en entrainant le réseau sur plusieurs GPU est Keras. Il suffit de modifier quelques lignes de codes pour passer au modèle multi-gpu. Lorsqu'on lance un entrainement sur un GPU, chaque itération se fait en entrainant plusieurs batch. Donc, si la taille du batch est de 32 et qu'un jeu d'entrainement comporte 1600 données, le réseau devra faire 50 descentes de gradient stochastiques par itération. Si maintenant on passe sur deux GPUs, et qu'on met la taille du batch à 64, celui-ci sera divisé par deux et chaque GPUs fera une descente de gradient stochastique sur son batch de taille 32. En parallèle, chacun des deux gradient effectuera donc 25 descentes de gradient, deux fois moins donc !

La grande force de ce parallélisme repose sur l'association CPU-GPUs. En effet, cette asso-

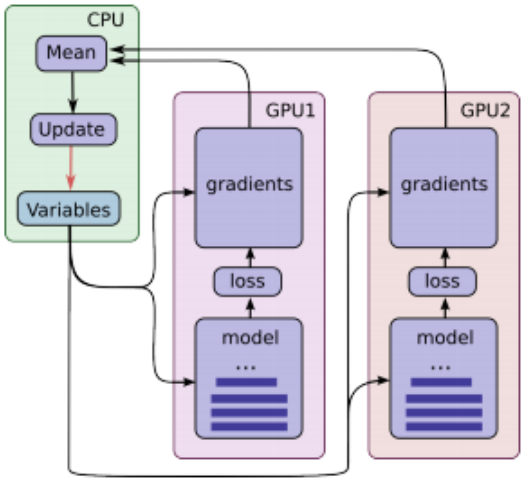


Figure III.6 – Parallélisme multi-GPUs

ciation va ainsi permettre de supprimer les communications inter-GPUs. Chaque GPU communiquera le résultat de sa descente de gradient stochastique au CPU du même nœud, qui va ensuite calculer rapidement la moyenne pour la retransmettre ensuite aux GPUs pour qu'ils puissent continuer l'apprentissage sur le batch suivant. La partie de communication est donc vraiment réduite si on passe de 2 GPUs 'a 4, 6, 10 etc. TensorFlow, qui est le backend sur lequel s'appuie Keras, promet un speed up linéaire, c'est-à-dire les temps d'exécutions se divisent par le nombre de GPUs s´sélectionnés. Ceci est vrai dans le cas où les GPUs sont identiques, car lors de la communication du n-ième batch, si un GPU termine avant , le CPU devra attendre que les autres terminent pour pouvoir ensuite faire la moyenne des poids reçus.

3.1.2 Entrainer le réseau sur plusieurs GPUs multi-machines

L'utilisation de plusieurs GPUs accélère le processus d'entrainement. Cependnat, le speed-up reste limité au nombre de GPUs existants sur la machine elle-même. C'est pour ça, une nouvelle approche de parallélisme qui consiste à utiliser des GPUs sur des machines différentes a été créée. Toutefois, cette fonctionnalité n'est pas supportée par Keras.

3.2 Génération des données

3.2.1 Data Generator

La méthode classique par laquelle on commence avant d'entrainer un réseau consiste à charger toutes les données dans la mémoire dans un seul coup. Cependant, ce chargement peut engendrer une surconsommation de la mémoire, surtout si la base est géante. En effet, nous

devons garder en l'esprit, que dans la plupart des cas, même les machines les plus puissantes ne disposent pas d'espace mémoire suffisant pour tous sauvegarder et par la suite assurer un bon fonctionnement du système. C'est pour cette raison, qu'il faut trouver d'autres moyens pour accomplir cette tâche efficacement. D'où l'introduction du concept de « Data-Generator » qui consiste à génrérer des paquets de données sur plusieurs cœurs en temps réel et de les intégrer immédiatement au modèle d'apprentissage.
L'utilisation du framework keras nous permet de mettre en place une telle fonctionnalité.

3.2.2 LRU cache

Avant tout, notez bien que dans ce projet les données utilisées sont des images en 3D. Chaque image peut présenter un ou plusieurs nodules. Ainsi, l'apprentissage se fait sur plusieurs itérations dont chacune le programme relit les même données de nouveau à partir de la mémoire et les intègre dans le réseau etc. Cela fonctionne, mais peut être lent. D'où la nécessité de l'utilisation d'un cache présentant un stockage rapide et assurant un accès direct. L'algorithme de LRU (Last Recently Used) cache consiste à allouer n cases pour stocker les données. Si toutes ces cases sont pleines, nous commençons par vider la case la plus ancienne pour la nouvelle donnée.

4 Classification des données déséquilibrées

Comme cité précédemment, avant d'entamer la modélisation, il faut commencer par une analyse rapide des données. Il arrive souvent dans les cas de classification en Deep Learning que l'une des classes soit minoritaire par rapport à la population globale. Dans ce cas, l'histogramme de la base de données présente un déséquilibrage entre la classe des négatifs et la classe des positifs. Cela peut être un problème car la plupart des algorithmes de classification se basent sur l'exactitude (ou l'accuracy) pour construire leurs modèles. Voyant que la grande majorité des observations appartient à la même catégorie, nous risquons de nous retrouver avec un modèle peu intelligent qui va toujours prédire la classe dominante. Or, dans le domaine médical, c'est la classe minoritaire présentant la maladie qui nous intéresse.
En effet, nous distinguons deux types d'approches : Les approches algorithmiques et les approches d'échantillonnage. Mais avant de décrire ces deux approches, il faut préciser la métrique qui correspond bien aux objectifs du projet pour valider les modèles obtenus.
L'erreur classique dans le cas de classes déséquilibrées est de croire que le modèle est performant parce qu'il possède une exactitude proche de 'un'. Ce qui n'est pas le cas car ça reflète que la bonne prédiction de la classe dominante. C'est pour cela, la métrique la plus adaptée est l'AUC

ROC ou le F1-score. Tout dépond de l'application.

4.1 Stratégies d'échantillonnage

Les stratégies d'échantillonnage visant à rééquilibrer les données. Ces méthodes peuvent par ailleurs être utilisées avec n'importe quelle méthode d'apprentissage supervisé. Ces techniques sont mis en œuvre soit en faisant un sous-échantillonnage, en enlevant des données de la classe majoritaire, soit en faisant de sur-échantillonnage, en rajoutant des nouvelles données dans la classe minoritaire. Il est toujours plus simple de supprimer des données que d'en rajouter. Cependant, le sous-échantillonnage intègre le risque de supprimer des individus représentatifs pour l'apprentissage. Ainsi, si nous n'avons pas pas beaucoup de données, nous pouvons difficilement en jeter une partie. Dans ce cas, deux options sont possibles :
- Collecter plus de données (en faisant plus de mesures ou en prenant un échantillon des années précédentes par exemple) ce qui n'est pas toujours évident et peut être couteux.
- Synthétisez de nouvelles données à partir des observations existantes en appliquant des techniques d'augmentation de données (voir II.4.d)
Notez bien que le sur-échantillonnage peut entrainer des problèmes de sur-apprentissage.

4.2 Stratégies algorithmiques

Nous avons vu comment adapter nos données à nos modèles mais il est également possible de faire l'inverse et d'adapter nos modèles à nos données !
En effet, l'implémentation **Scikit Learn** permet à ses utilisateurs d'attribuer plus de poids à certaines observations via le paramètre **class_ weight**. Il se présente sous la forme d'un dictionnaire ou d'une String. Si nous écrivons par exemple class_ weight = { 0 : 1, 1 : 5 } nous indiquons à l'algorithme que les valeurs positifs comptent 5 fois plus que les valeurs négatives. Nous pouvons ainsi passer la chaine « balanced » à ce paramètre, et dans ce cas l'attribution des poids se fera automatiquement en fonction de la proportion des classes. Ainsi, nous citons la fonction de perte 'focal-loss' [29] qui remplace la fonction de perte 'cross-entropy'. Cette fonction a prouvé sa robustesse vis-à-vis le problème de déséquilibrage des données pour la classification en pénalisant les exemples facilement classés tout en suivant la formule 1 :
$\mathrm{FL}(\mathrm{p}_t) = -(1-p_t)^{\gamma} log(p_t)$

La figure III.7 illustre l'effet du facteur gamma sur la perte : Finalement, il est difficile de dire à l'avance quelle solution sera la plus adaptée à un problème donné. Cela nécessite la réalisation plusieurs tests pour privilégier la solution qui semble la meilleure.

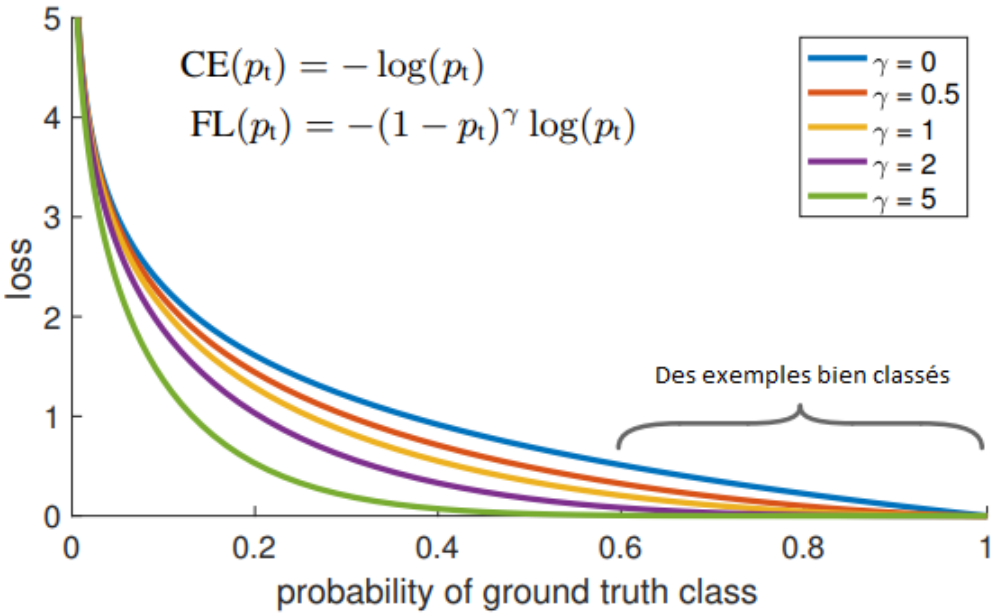


Figure III.7 – L'effet du gamma sur la fonction Focal-Loss

Conclusion

Dans ce chapitre, nous avons présenté des différentes méthodes utilisées pour améliorer les performances du modèle que ce soit en termes de temps ou d'efficacité.

Chapitre IV

Implémentation et discussion des résultats

Introduction

Après avoir étudié dans le deuxième chapitre les différentes approches utilisées pour le problème de classification du cancer du poumon, et dans le troisième chapitre les méthodes d'optimisation possibles, nous allons citer dans ce chapitre le meilleur modèle obtenu selon la métrique AUC.

1 Outils utilisés

L'engouement actuel pour le Deep Learning s'alimente des progrès en calcul GPU et des librairies open source de machine learning. Plusieurs frameworks sont utilisés pour implémenter des réseaux de neurones, et la majorité supporte le langage Python. Parmi les célèbres figurent TensorFlow, Keras, Lasagne, Caffee du computer vision lab de Berkeley en Matlab/Python4 et Torch7 utilisé par Google, Facebook, Twitter se distinguant par son environnement en Lua. L'utilisation de GPUs, possible grâce aux librairies citées, peut s'avérer alors nécessaire pour réduire les temps de calculs.

Dans ce projet, nous nous intéressons essentiellement aux frameworks keras et TensorFlow :

TensorFlow : C'est un framework de programmation pour le calcul numérique qui a été rendu Open Source par Google en Novembre 2015. Depuis son release, TensorFlow n'a cessé de gagner en popularité, pour devenir très rapidement l'un des frameworks les plus utilisés pour le Deep Learning et donc les réseaux de neurones. Aujourd'hui, les principaux produits de Google sont

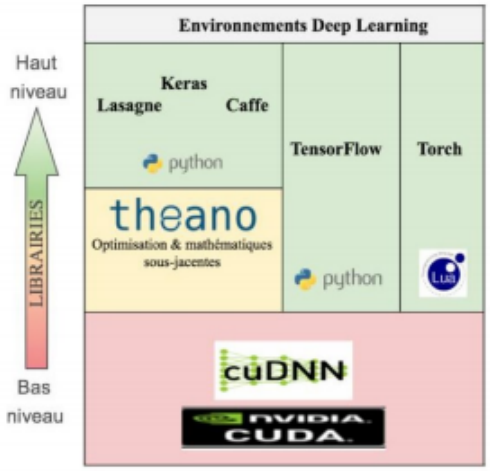


Figure IV.1 – Aperçu de l'environnement de Deep Learning

basés sur TensorFlow : Gmail, Google Photos, Reconnaissance de voix...

Keras : C'est une API de réseaux de neurones de haut niveau, écrite en Python et capable de fonctionner sur TensorFlow ou Theano. Il a été développé en mettant l'accent sur l'expérimentation rapide. Être capable d'aller de l'idée à un résultat avec le moins de délai possible est la clé pour faire de bonnes recherches. En 2017, l'équipe TensorFlow de Google a décidé de soutenir Keras dans la bibliothèque principale de TensorFlow. Son fondateur Chollet a expliqué que Keras a été conçue comme une interface plutôt que comme un cadre d'apprentissage end to end. Il présente un ensemble d'abstractions de niveau supérieur et plus intuitif qui facilitent la configuration des réseaux neuronaux indépendamment de la bibliothèque informatique de backend.

Anaconda : C'est une distribution Python. A son installation, Anaconda installera Python ainsi qu'une multitude de packages. Cela nous évite de nous ruer dans les problèmes d'incompatibilités entre les différents packages. Ainsi, Anaconda propose un outil de gestion de packages appelé Conda. Ce dernier permettra de mettre à jour et installer facilement les librairies et par la suite il nous permet de créer un environnement virtuel spécifique à un projet donné.

Git : Tout le travail est sauvegardé sur un serveur local « git » qui permet la gestion des versions.

2 Configuration matérielle

La configuration du matériel utilisé dans notre implémentation est :

— Un PC Intel (R) Core (TM) i7-3770 CPU 3.40 GHZ

— RAM de taille 8GO
— Carte graphique Nvidia Corporation GP106 GeForce GTX1060 6GB
— Système d'exploitation Linux Ubuntu version 16.04

3 Base de données

LIDC/IDRI :

Il s'agit d'une ressource internationale accessible sur le Web afin de développer, améliorer et évaluer les systèmes CAD pour la détection et le diagnostic du cancer du poumon. Sept centres universitaires et huit sociétés d'imagerie médicale ont collaboré pour la création de cet ensemble de données contenant 1018 cas. Chaque sujet comprend des images d'un scanner CT thoracique clinique et un fichier XML associé qui stocke les résultats d'un processus d'annotation d'image en deux phases effectué par quatre radiologistes expérimenté. Au cours de la phase initiale de lecture, chaque radiologiste a examiné indépendamment chaque tomodensitogramme et a marqué les lésions appartenant à l'une des trois catégories ("nodule> ou = 3 mm", "nodule <3 mm" et "non-nodule> ou = 3 mm"). Au cours de la phase suivante, chaque radiologiste a examiné de manière indépendante ses propres marques, ainsi que les marques anonymes des trois autres radiologistes, afin de rendre un avis final.

LUNA16 :

Luna16 est construite suite au challenge intitulé 'Lung Nodule Analysis' en 2016. La base est disponible sur le Web. 888 CT scans accompagnées par un fichier contenant les coordonnées de chaque tumeur ainsi que sa nature (bénigne ou maligne) sont extraites à partir de LIDC-IDRI. Ces coordonnées présentent le résultat de plusieurs détecteurs de nodules. L'étude de ce projet n'introduit que la base de données LUNA16.

La figure IV.2 présente des tranches 2D d'un seul patient extraites à partir de différentes localisations.

4 Réalisation du modèle

4.1 Préparation des données

Les images CT scans peuvent contenir une ou plusieurs tumeurs à la fois. Donc la première phase consiste à extraire ces tumeurs pour l'utiliser comme entrée au réseau. En effet, les 888 CT scans présentent 736975 tumeurs en 3D, ce qui nécessite une mémoire géante pour les charger en même temps. Or, les ressources disponibles rendent cette mission quasiment impossible

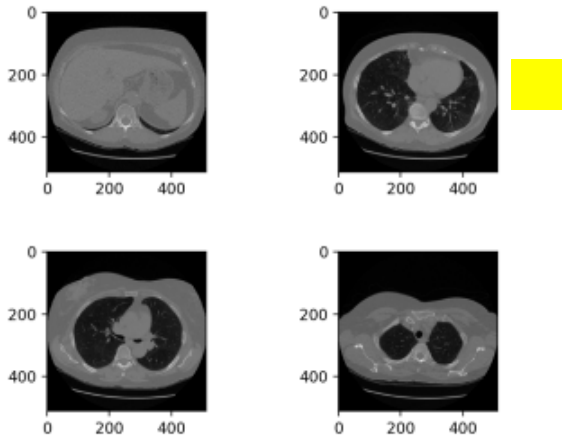


Figure IV.2 – Des tranches 2D extraites de différentes localisations d'un seul patient [2]

car même si elle ne génère pas une erreur de mémoire, le temps d'exécution peut atteindre 9 heures ou plus. D'où l'introduction du data generator qui rend cette procédure de 50 à 100 fois plus rapide que la méthode classique. Nous introduisons ainsi le LRU cache. En fait, après plusieurs tests, nous avons constaté que, pour un batch de taille 32, il nous faut 6.865s pour le charger. L'utilisation de LRU cache au moment d'exécution (en temps réel) réduit ce temps pour atteindre 0.1s/batch. Les cubes des tumeurs extraites doivent ensuite être normalisés.

4.2 Equilibrage des données

De 736975 tumeurs, seulement 1557 présentent des exemples positifs, donc la prédiction de la classe majoritaire devient plus évidente, ce qui ignore la classe 1. Pour remédier à ce problème, nous avons mis en place plusieurs techniques d'augmentation des données dont nous citons la rotation, la translation, l'effet miroir et le sur-échantillonnage. Ainsi, nous avons utilisé l'apprentissage pondéré en fixant le paramètre class-weight à 100 pour les exemples positifs et 1 pour le reste.

4.3 Architecture du réseau

Au cours de nos expérimentations, nous avons créé plusieurs modèles avec des architectures différentes et finalement, nous avons conservé que l'architecture que nous a donné les meilleurs résultats. Dans ce qui suit nous présentons l'architecture finale utilisée :
Pour construire notre modèle, nous avons utilisé les réseaux de neurones convolutifs. En effet, le modèle présenté dans la figure IV.3 est composé principalement de 4 stages. Pour les 3 premièrs, chaque stage est composée de deux couches de convolution 3D suivies par deux couches de normalisation et deux couches de maxpooling 3D. Nous avons ajouté à la fin de chaque étape

une couche de Dropout afin d'éviter le phénomène de sur-apprentissage. La dernière étape est composée de trois couches fully connected suivies par des couches de Dropout.
En entrée, nous avons utilisé des images de taille 32*32*32 ou de taille 64*64*64 pour varier les résultats. Notez bien qu'il faut ignorer la dernière couche maxPooling dans le cas des images de taille 32*32*32. Au moment de l'exécution, l'image passe d'abord par la première couche de convolution. Cette couche est composée de 32 filtres de taille 3*3*3. Chacune des couches de convolution est suivie d'une fonction d'activation ReLU permettant de forcer les neurones à retourner des valeurs positives. Les 32 feature maps obtenues auparavant présentent l'entrée de la couche MaxPooling composée d'un filtre de taille 2*2*2. Ensuite, le meme bloc sera répété. En passant d'un stage à un autre, nous doublons le nombre de filtres des couches de convolution. À la sortie, les feature maps résultantes seront passé par une couche fully connected. Les deux premières couches ont chacune 512 neurones où la fonction d'activation utilisée est le ReLU, et la troisième couche applique la fonction d'activation sigmoid présentant en sortie la classe de nodule.
La figure IV.4 présente un résumé de la configuration du modèle 3D-DCNN pour une image de taille 32*32*32

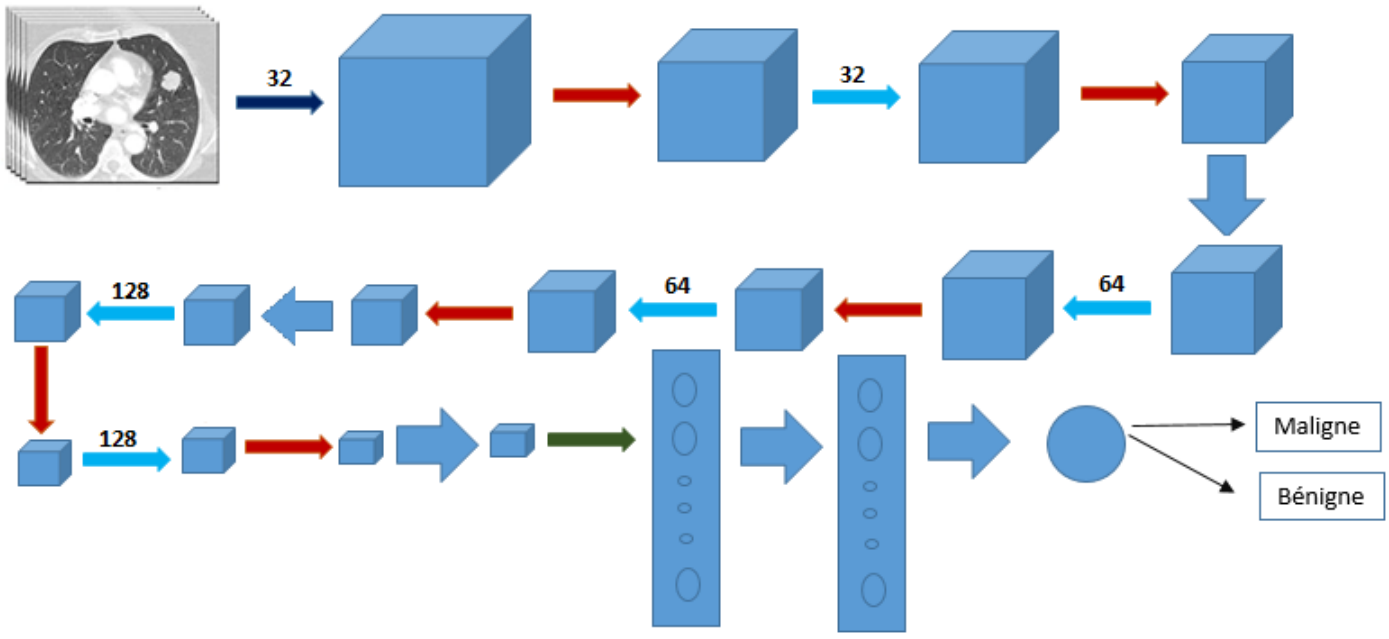


Figure IV.3 – Architecture du modèle 3D-DCNN

Layer (type)	Output Shape	Param #
input_3 (InputLayer)	(None, 32, 32, 32, 1)	0
batch_normalization_15 (Batc	(None, 32, 32, 32, 1)	4
conv3d_13 (Conv3D)	(None, 32, 32, 32, 32)	896
batch_normalization_16 (Batc	(None, 32, 32, 32, 32)	128
max_pooling3d_11 (MaxPooling	(None, 16, 16, 16, 32)	0
conv3d_14 (Conv3D)	(None, 16, 16, 16, 32)	27680
batch_normalization_17 (Batc	(None, 16, 16, 16, 32)	64
max_pooling3d_12 (MaxPooling	(None, 8, 8, 8, 32)	0
dropout_11 (Dropout)	(None, 8, 8, 8, 32)	0
conv3d_15 (Conv3D)	(None, 8, 8, 8, 64)	55360
batch_normalization_18 (Batc	(None, 8, 8, 8, 64)	32
max_pooling3d_13 (MaxPooling	(None, 4, 4, 4, 64)	0
conv3d_16 (Conv3D)	(None, 4, 4, 4, 64)	110656
batch_normalization_19 (Batc	(None, 4, 4, 4, 64)	16
max_pooling3d_14 (MaxPooling	(None, 2, 2, 2, 64)	0
dropout_12 (Dropout)	(None, 2, 2, 2, 64)	0
conv3d_17 (Conv3D)	(None, 2, 2, 2, 128)	221312
batch_normalization_20 (Batc	(None, 2, 2, 2, 128)	8
max_pooling3d_15 (MaxPooling	(None, 1, 1, 1, 128)	0
conv3d_18 (Conv3D)	(None, 1, 1, 1, 128)	442496
batch_normalization_21 (Batc	(None, 1, 1, 1, 128)	4
dropout_13 (Dropout)	(None, 1, 1, 1, 128)	0
flatten_3 (Flatten)	(None, 128)	0
fc1 (Dense)	(None, 512)	66048
dropout_14 (Dropout)	(None, 512)	0
fc2 (Dense)	(None, 512)	262656
dropout_15 (Dropout)	(None, 512)	0
output (Dense)	(None, 1)	513

Total params: 1,187,873
Trainable params: 1,187,745
Non-trainable params: 128

Figure IV.4 – Configuration du modèle 3D-DCNN pour une image de taille 32*32*32

4.4 Initialisation des paramètres

Dans le domaine de deep learning, l'initialisation des poids joue un rôle important et peut créer une différence en assurant une convergence du modèle dans un temps raisonnable. En effet, assigner des poids au réseau avant de commencer l'apprentissage semble être un processus aléatoire puisque nous ne savons pas le produit du modèle avec nos données. Une bonne façon

est d'assigner les poids à partir d'une distribution gaussienne. Évidemment, cette distribution aurait une moyenne nulle et une variance finie. Cela nous aide à empêcher le signal d'exploser à une valeur élevée ou à disparaître à zéro. En d'autres termes, nous devons initialiser les poids de telle sorte que la variance reste constante tout au long du réseau. Ce processus est noté l'initialisateur Xavien (connu aussi sous le nom « Glorot initialization »).

4.5 La fonction de perte

Comme cité auparavant, le problème majeur de ce système est le déséquilibrage des données. D'où l'introduction de la fonction de perte « Focal loss » [29] qui a remplacé la fonction entrepie croisée (cross-entropy). Cette fonction est formée de telle sorte qu'elle empêche le classifier de considérer que les exemples négatifs (facile à prédire) pendant la phase d'apprentissage.

4.6 Optimisation du calcul

Contrairement à TensorFlow, Keras n'offre pas la fonctionnalité de paralléliser l'entrainement sur plusieurs machines. Cependant, l'implémentation de data-generator invoque l'utilisation de la fonction fit_ generator. Cette fonction nous permet d'introduire la fonctionnalité de parallélisme des GPUs sur une même machine en fixant le paramètre use-multiprocessing à « True ». Dans ce cas, le nombre des cœurs utilisés est fixé par le paramètre « workers ». Ainsi, nous avons contribué à améliorer le temps d'exécution des GPUs eux-mêmes grâce à une approche de lecture des données par batch. En d'autres termes, au lieu d'accéder de façon répétitive à la mémoire pour lire une seule donnée, nous avons augmenté la quantité des données (i.e le nombre des batchs) chargées à la fois par le GPU. La mise en place de cette fonctionnalité se fait en attribuant une valeur au paramètre « maxqueue size ».

4.7 Optimisation du traitement

Après avoir créé le modèle, le compiler et l'entrainer, il faut trouver les bonnes valeurs des hyper paramètres qui nous permettent d'avoir un modèle qui maximise la métrique AUC. Les tests seront faits sur l'architecture illustrée auparavant. Les poids sont initialisés avec Xavien. Le nombre d'itérations sera fixé selon le test et la convergence du modèle. La taille de batch est fixée à 16. La taille du cache RLU est mise à 1. La fonction de perte est la « focal loss ».

5 Tests et résultats

5.1 Données en entrée

Nous avons commencé par modifier la taille de l'image en entrée. Comme illustré dans la figure ci-dessous, les images de taille 32*32*32 présentant le modèle 1 permettent d'atteindre 96.86% d'AUC après 100 itérations versus 97,50 d'AUC pour le deuxième modèle.
Le temps d'exécution pour le deuxième modèle est 1288043s (environ deux semaines) alors que

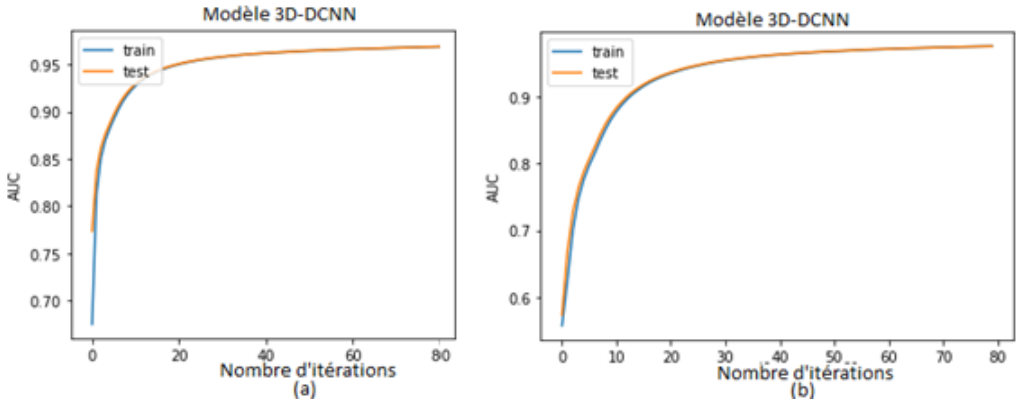


Figure IV.5 – Modèle1 avec des images en entrée de taille 32*323*32 ; (b) Modèle2 avec des images en entrée de taille 64*64*64

le premier modèle s'exécute dans seulement 269197s (environ 3 jours). Dans ce qui suit, nous allons considérer que les images de taille 32*32*32 puisque le deuxième modèle présente une consommation abusive des ressources et un gaspillage du temps.

5.2 Taux d'apprentissage

Les chercheurs considèrent que le taux d'apprentissage (ou The learning rate) est le paramètre le plus important pour la convergence du modèle. Si sa valeur est trop grande, les pas de gradient risquent de passer un éventuel minimum intéressant et augmenter l'erreur au lieu de la diminuer. Le cas contraire rend la convergence des poids trop lente. L'idée est de chercher une valeur convenable pour l'initialiser, puis l'ajuster au cours de traitement par le paramètre « decay ». Ce paramètre sert à réduire le taux d'apprentissage après chaque itération. Nous avons fixé le decay à 10-6 et nous avons choisi l'optimisateur SGD. Les deux figures IV.6 et IV.7 présentent les résultats obtenus pour un taux d'apprentissage fixé à 0.0001 et 0.00001 respectivement.
Puisque la valeur de l'AUC empire de plus en plus pour un taux d'apprentissage égal à 0.0001, nous avons abandonné l'entrainement après 30 itérations. D'ailleurs la courbe de perte montre

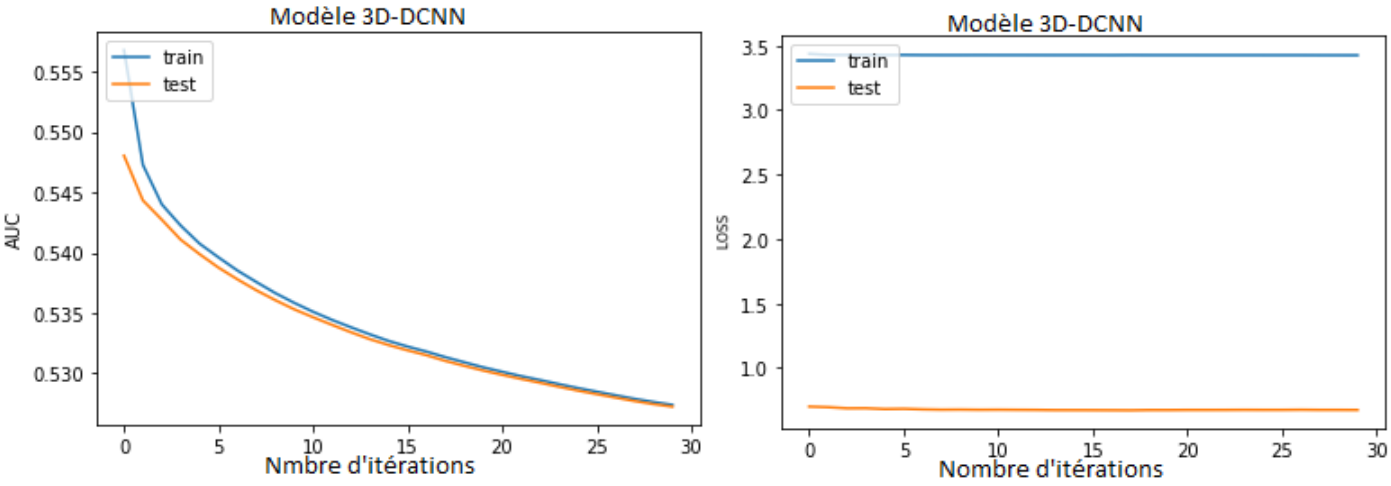


Figure IV.6 – lambda=0.0001

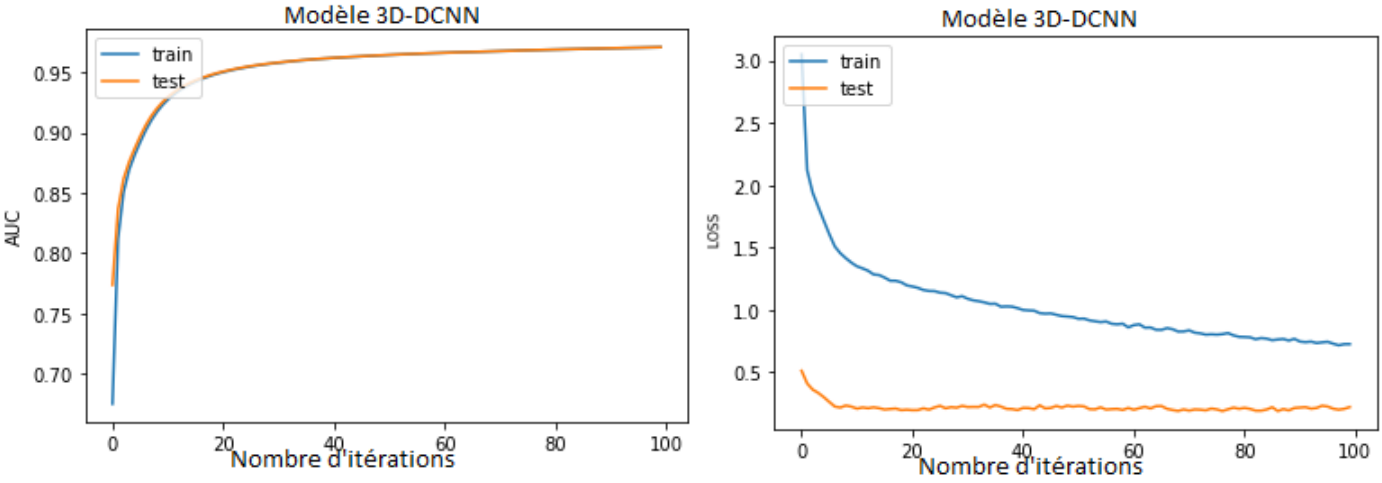


Figure IV.7 – Taux d'apprentissage=0.00001

que le réseau ne minimise pas l'erreur. Dans le reste de cette étude, nous considérons un taux d'apprentissage égal à 0.00001.

Nous avons ainsi testé les performances du modèle avec l'optimisateur ADAM. Cependant, le modèle n'évolue pas et l'AUC reste à 50% pendant longtemps. Notez bien, que cette valeur reflète le non apprentissage du modèle puisque le taux de prédiction des exemples positifs et celui de prédiction des exemples négatifs sont égaux.

5.3 Dropout

Nous avons testé un modèle sans ajouter des couches de dropout. Après 70 itérations, nous avons obtenu 95,32%. Nous notons aussi un sur-apprentissage après environ 50 itérations où le taux d'erreur de test dépasse celui d'entrainement. Dès le début, le taux d'erreur de test commence à augmenter progressivement et la courbe diverge alors que l'erreur de l'apprentissage diminue au fil de temps. Ce phénomène est présenté dans la figure IV.8.

Ensuite, nous avons inclus la technique de régularisation « dropout » afin de remédier à ce

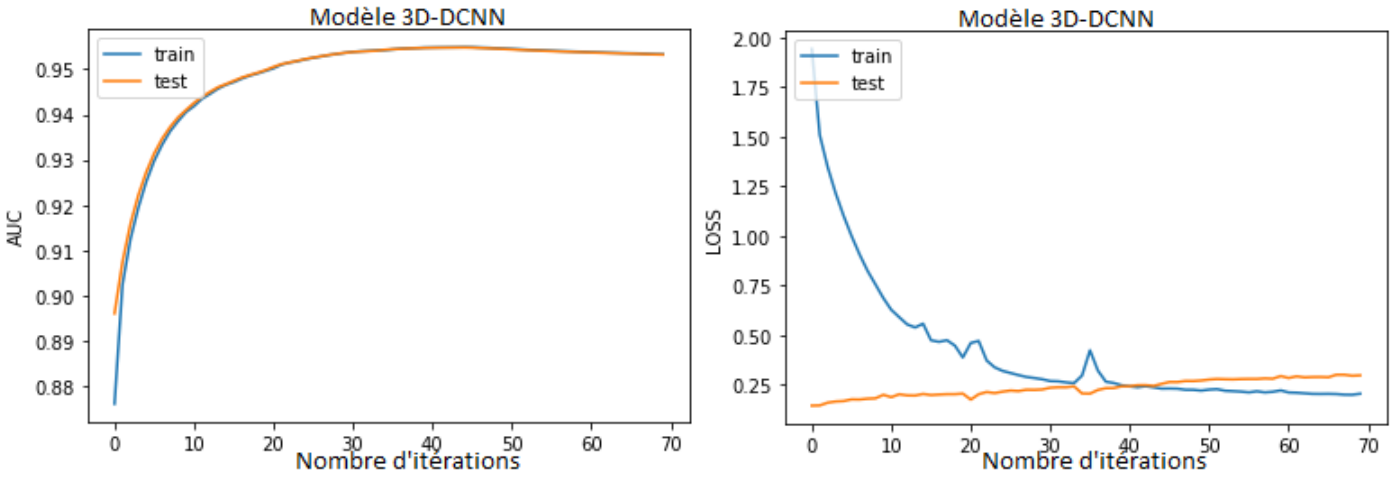


Figure IV.8 – Modèle sans dropout

problème. Le taux de rejet diffère d'un niveau de réseau à un autre. Après plusieurs tests, nous avons attribué les probabilités de rejet 0.1, 0.2, 0.3, 0 .4 et 0.4 respectivement aux couches de dropout comme illustré dans l'architecture du modèle présenté auparavant. Ceci nous mène aux résultats présentés dans la figure IV.9 : Conclusion : Pour parvenir à un tel degré de performance qui se manifeste par 97% de AUC, nous avons utilisé les données de tomodensitométrie des personnes atteintes d'un cancer du poumon et en comparant des tumeurs cancéreuses par d'autres bénignes.

5.4 Résultat final

Après avoir tester le modèle en modifiant à chaque fois un paramètre et en changeant une technique et ajoutant d'autres, nous avons construit un modèle permettant d'avoir 97,84% de AUC après 100 itérations. La matrice de confusion obtenue est présentée dans le tableau ci-dessous :[[110000 661][1779 520]].Le code utilisé est décrit en détails dans l'annexe. La valeur de AUC obtenue présente une amélioration des performances pour les systèmes de détection

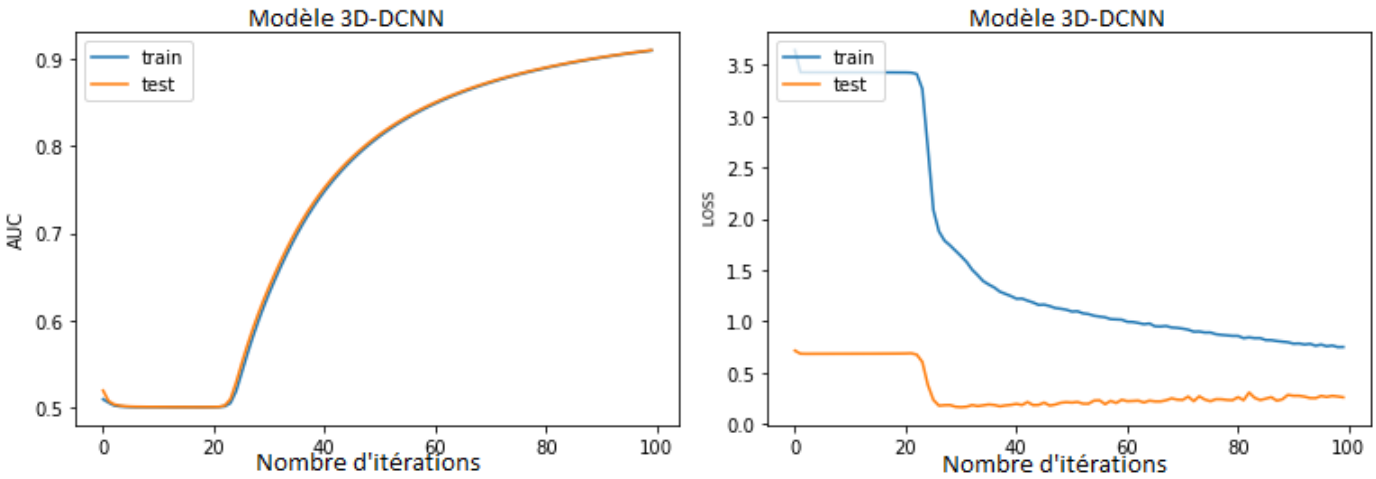


Figure IV.9 – Modèle avec Dropout

des tumeurs pulmonaires, reste à améliorer la matrice de confusion qui est sensible aux seuils choisis.

Conclusion

Les réseaux de neurones convolutifs présentent un certain nombre de limitations. En premier lieu, les hyper paramètres du réseau sont difficiles à évaluer a priori et présentent des éléments cruciaux et essentiellement déterminés par une bonne intuition ou par une succession de tests. La distribution des données utilisées lors de la phase d'apprentissage est également un élément déterminant. Dans ce chapitre, nous avons proposé un modèle de classification du cancer du poumon, et les résultats obtenus.

Conclusion Générale et Perspectives

Grâce au Deep Learning, l'avenir de l'intelligence artificielle est prometteur. Dans ce projet, les réseaux de neurones convolutifs ont montré leur performance pour le diagnostic et l'imagerie médicale. Malgré leur aspect boite noire, il reste essentiel d'en comprendre les principes. En effet, nous avons passé beaucoup de temps à étudier et analyser les articles liés au sujet de classification des images CT scans pour pouvoir finalement concevoir notre modèle et aboutir à ces résultats. Cela permet de combiner plusieurs techniques déjà proposées pour remédier à la problématique de cancer du poumon. C'est pour cela, la phase de prétraitement que nous avons appliqué avait un impact direct sur l'amélioration du modèle L'ajout des exemples vrai positifs a beaucoup joué sur la qualité de l'apprentissage et donc sur la prédiction du réseau. Une fois cela compris, jouer sur la profondeur du réseau et les hyper paramètres permet d'optimiser le réseau.

L'expérience vécue le long de ce stage de fin d'études a été très enrichissante aussi bien au niveau relationnel en côtoyant des compétences qualifiées au quotidien qu'au niveau technologique en manipulant des notions de réseaux de neurones convolutifs.

Nous rappelons que les résultats obtenus doivent encore être validés cliniquement par la biopsie, auprès de vastes populations de patients. Cependant, nous sommes très optimistes vis-à-vis cette étude puisqu'un tel appui de l'intelligence artificielle pourrait non seulement diagnostiquer plus précocement la maladie et limiter les procédures inutiles, mais aussi "améliorer la prise en charge et les résultats des patients atteints de cancer du poumon".

Il faudra cependant attendre encore plusieurs années avant que les études réalisées en intelligence artificielle ne soient transférées à la pratique dans nos hôpitaux, estime le Dr Tang : « Nous en sommes à la preuve de concept. Ça fonctionne sur un jeu de données. Si on prend les images de scans réalisées avec différentes techniques et doses de colorant, provenant de manufacturiers ou d'hôpitaux différents, est-ce que l'algorithme va fonctionner aussi bien ? Il y a encore plusieurs défis à relever pour appliquer ces algorithmes à large échelle. On est encore

dans le registre de la recherche et du développement. Nous devrons valider auprès d'une large population, dans différents scénarii d'acquisitions d'images pour confirmer la robustesse de cet algorithme. » Et cette grande question qui soulève plusieurs débats : est-ce que ces nouvelles avancées en intelligence artificielle vont remplacer l'humain en médecine ? « Non, au contraire. L'intelligence artificielle va nous permettre de réaliser des tâches qu'on ne peut pas faire actuellement parce que ça prend trop de temps. Ça nous donne des outils additionnels. On n'élimine pas d'emplois, on vient ajouter de nouvelles capacités pour analyser un grand nombre d'images », conclut le Dr An Tang.

Bien que les principaux objectifs de notre projet soient atteints, il n'y a jamais d'œuvre parfaite. Le travail sur des tumeurs en 3D nécessite un algorithme de segmentation en 3D aussi. Les études faites qui concernent ce sujet, se base encore sur des approches en 2D ce qui limite les performances du système. Donc, ce travail reste ouvert à l'extension d'une telle approche.

Bibliographiques

[1] Qi Dou, Hao Chen, Lequan Yu, Jing Qin, and Pheng-Ann Heng. *Multilevel contextual 3-d cnns for false positive reduction in pulmonary nodule detection.* IEEE Transactions on Biomedical Engineering **64**(7), 1558–1567 (2016). v, 6, 20

[2] Albert Chon, Niranjan Balachandar, and Peter Lu. *Deep convolutional neural networks for lung cancer detection.* Standford University (2017). vi, 36

[3] Canadian Cancer Society. Canadian Cancer Statistics. http://www.cancer.ca/fr-ca/cancer-information/cancer-type/lung/statistics/?region=pe, (2017). 4

[4] Institut National du cancer. *Les traitements des cancers du poumon.* (2017). 5

[5] Frank Rosenblatt. *The perceptron : a probabilistic model for information storage and organization in the brain.* Psychological review (1958). 6

[6] Christopher M Bishop et al. *Neural networks for pattern recognition.* Oxford university press (1995). 7

[7] I Goodfellow, Y Bengio, and A Courville. *Deep learning, 2016, book in preparation for mit press.* (2014). 8, 11, 12, 13

[8] Geoffrey Hinton, Nitish Srivastava, and Kevin Swersky. *Neural networks for machine learning lecture 6a overview of mini-batch gradient descent.* Cited on (2012). 8

[9] Jürgen Schmidhuber. *Deep learning in neural networks : An overview.* Neural networks (2015). 10

[10] Pierre-Alain Jachiet Julien Krywyk. Classification d'images : les réseaux de neurones convolutifs en toute simplicité. https://blog.octo.com/classification-dimages-les-reseaux-de-neurones-convolutifs-en-toute-sim (2016). 10

[11] F.-F. Li and A. Karpathy. Stanford CS231n Course Materials,Convolutional neural networks. http://cs231n.github.io/convolutional-networks/., (2015). 10

[12] Ml practicum : Image classification. https://developers.google.com/machine-learning/practica/image-classification/convolutional-neural-networks, (2019). 12

[13] American Lung Association. Lung cancer factsheet. http://www.lung.org/lung-health-and-diseases/lung-diseaselookup/lung-cancer/resource-library/lung-cancer-fact-sheet.html, (2018). 13

[14] T Jia, H Zhang, and YK Bai. *Benign and malignant lung nodule classification based on deep learning feature*. Journal of Medical Imaging and Health Informatics **5**(8), 1936–1940 (2015). 16

[15] Cheng-Yuan Liou, Jau-Chi Huang, and Wen-Chie Yang. *Modeling word perception using the elman network*. Neurocomputing **71**(16-18), 3150–3157 (2008). 16

[16] Arnaud Arindra Adiyoso Setio, Francesco Ciompi, Geert Litjens, Paul Gerke, Colin Jacobs, Sarah J Van Riel, Mathilde Marie Winkler Wille, Matiullah Naqibullah, Clara I Sánchez, and Bram van Ginneken. *Pulmonary nodule detection in ct images : false positive reduction using multi-view convolutional networks*. IEEE transactions on medical imaging **35**(5), 1160–1169 (2016). 17

[17] Keelin Murphy, Bram van Ginneken, Arnold MR Schilham, BJ De Hoop, HA Gietema, and Mathias Prokop. *A large-scale evaluation of automatic pulmonary nodule detection in chest ct using local image features and k-nearest-neighbour classification*. Medical image analysis **13**(5), 757–770 (2009). 18

[18] Colin Jacobs, Eva M van Rikxoort, Thorsten Twellmann, Ernst Th Scholten, Pim A de Jong, Jan-Martin Kuhnigk, Matthijs Oudkerk, Harry J de Koning, Mathias Prokop, Cornelia Schaefer-Prokop, et al. *Automatic detection of subsolid pulmonary nodules in thoracic computed tomography images*. Medical image analysis **18**(2), 374–384 (2014). 18

[19] Arnaud AA Setio, Colin Jacobs, Jaap Gelderblom, and Bram van Ginneken. *Automatic detection of large pulmonary solid nodules in thoracic ct images*. Medical physics **42**(10), 5642–5653 (2015). 18, 19

[20] J-M Kuhnigk, Volker Dicken, Lars Bornemann, Annemarie Bakai, Dag Wormanns, Stefan Krass, and H-O Peitgen. *Morphological segmentation and partial volume analysis for volumetry of solid pulmonary lesions in thoracic ct scans*. IEEE transactions on medical imaging **25**(4), 417–434 (2006). 18

[21] Dan Cireşan, Ueli Meier, Jonathan Masci, and Jürgen Schmidhuber. *Multi-column deep neural network for traffic sign classification*. Neural networks **32**, 333–338 (2012). 19

[22] Adhish Prasoon, Kersten Petersen, Christian Igel, François Lauze, Erik Dam, and Mads Nielsen. Deep feature learning for knee cartilage segmentation using a triplanar convolutional neural network. In *International conference on medical image computing and computer-assisted intervention*, pages 246–253. Springer (2013). 20

[23] Holger R Roth, Le Lu, Ari Seff, Kevin M Cherry, Joanne Hoffman, Shijun Wang, Jiamin Liu, Evrim Turkbey, and Ronald M Summers. A new 2.5 d representation for lymph node detection using random sets of deep convolutional neural network observations. In *International conference on medical image computing and computer-assisted intervention*, pages 520–527. Springer (2014). 20

[24] missinglink.ai. Neural Network Bias : Bias Neuron, Overfitting and Underfitting. https://missinglink.ai/guides/neural-network-concepts/neural-network-bias-bias-neuron-overfitting-underfitting/#!#managingbiasandvariance, (2015). 23

[25] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press (2016). 25

[26] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. *Dropout : a simple way to prevent neural networks from overfitting*. The journal of machine learning research **15**(1), 1929–1958 (2014). 26

[27] Warren S Sarle. *Stopped training and other remedies for overfitting*. Computing science and statistics pages 352–360 (1996). 26

[28] Matteo Arvetti, Giuseppina Gini, and Michele Folgheraiter. Classification of emg signals through wavelet analysis and neural networks for controlling an active hand prosthesis. In *2007 IEEE 10th international conference on Rehabilitation Robotics*, pages 531–536. IEEE (2007). 26

[29] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988 (2017). 31, 39